

分类算法

2026 年 08 月

本章目录

01 前言
02 kNN 算法
03 朴素贝叶斯
04 决策树

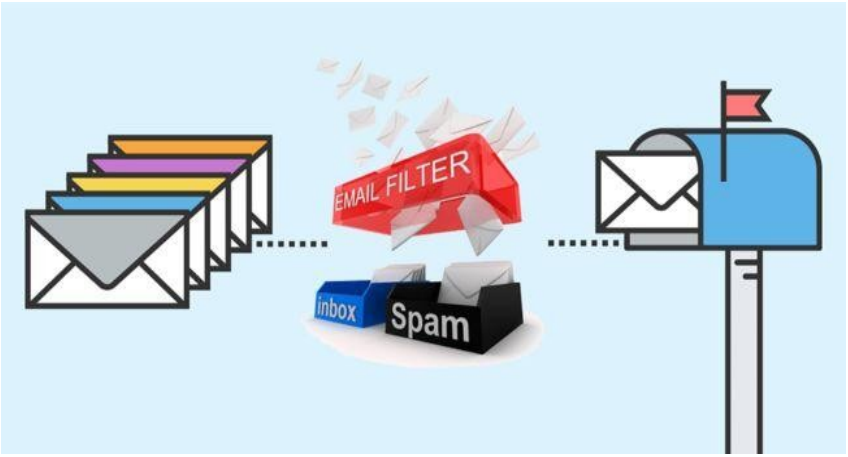
1. 前言

- 分类是监督学习的核心问题之一。在分类问题中，数据的输出属性是离散的。

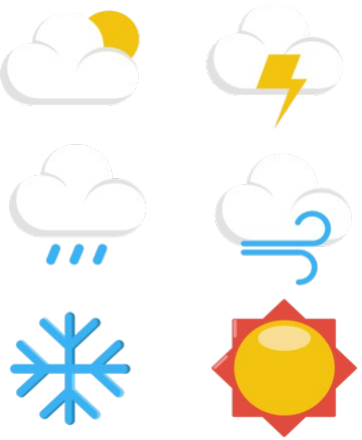
- 当分类的类别是两个时，为二分类问题；当分类的类别大于两个的时候，为多分类问题。

标签离散

邮件分类



天气预测



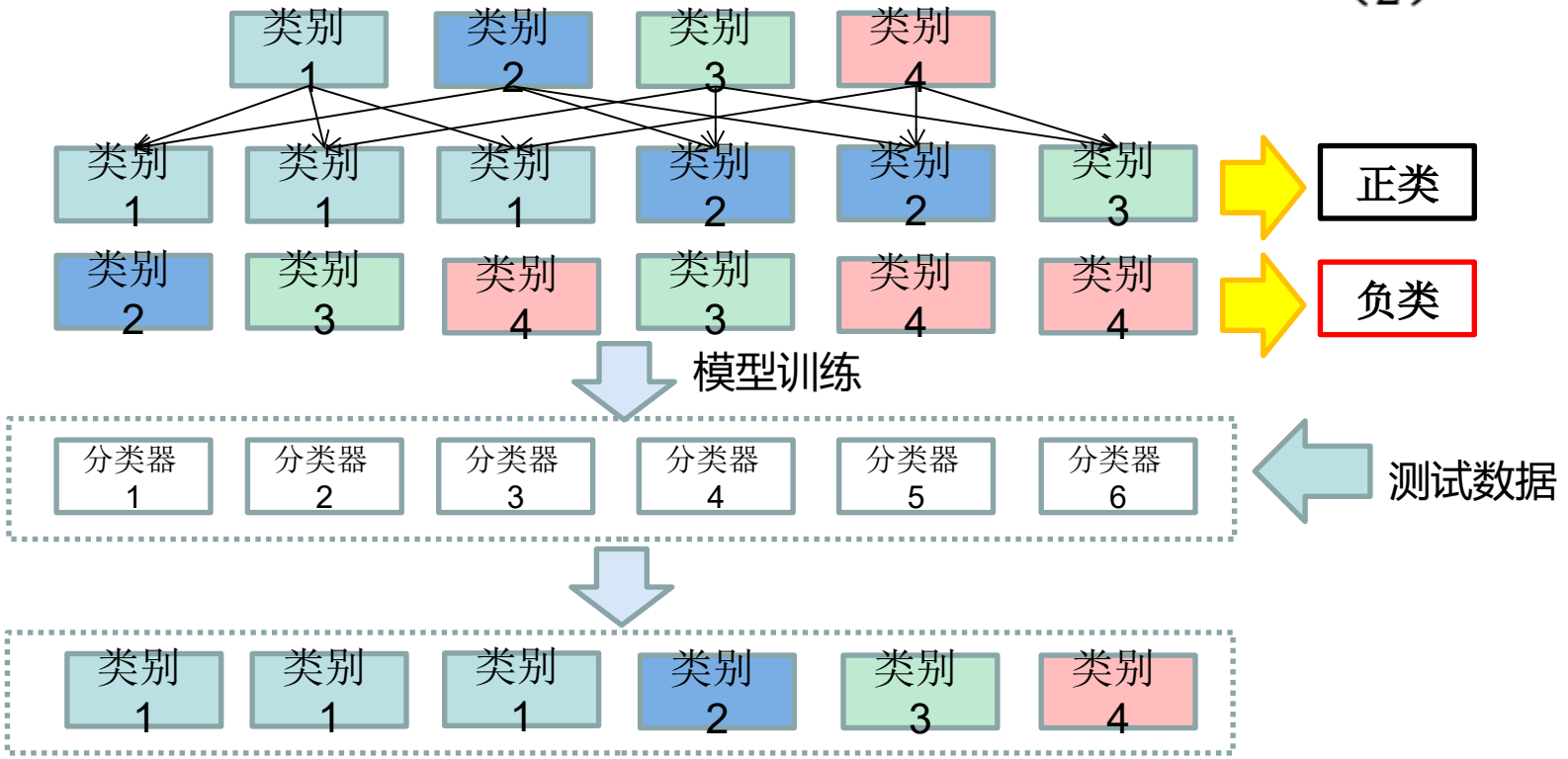
1、前言

- 二分类问题：在二分类问题中，预测输出变量只有两个值，一般情况下，我们用0来表示负类（注：有些书也用-1表示负类），用+1表示正类。对于这种二分类问题，我们只需要构建一个分类器即可。

- 多分类问题：在多分类问题中，输出预测变量则有多个值 $y = \{0,1,2,3,\ldots.\}$

1. 前言

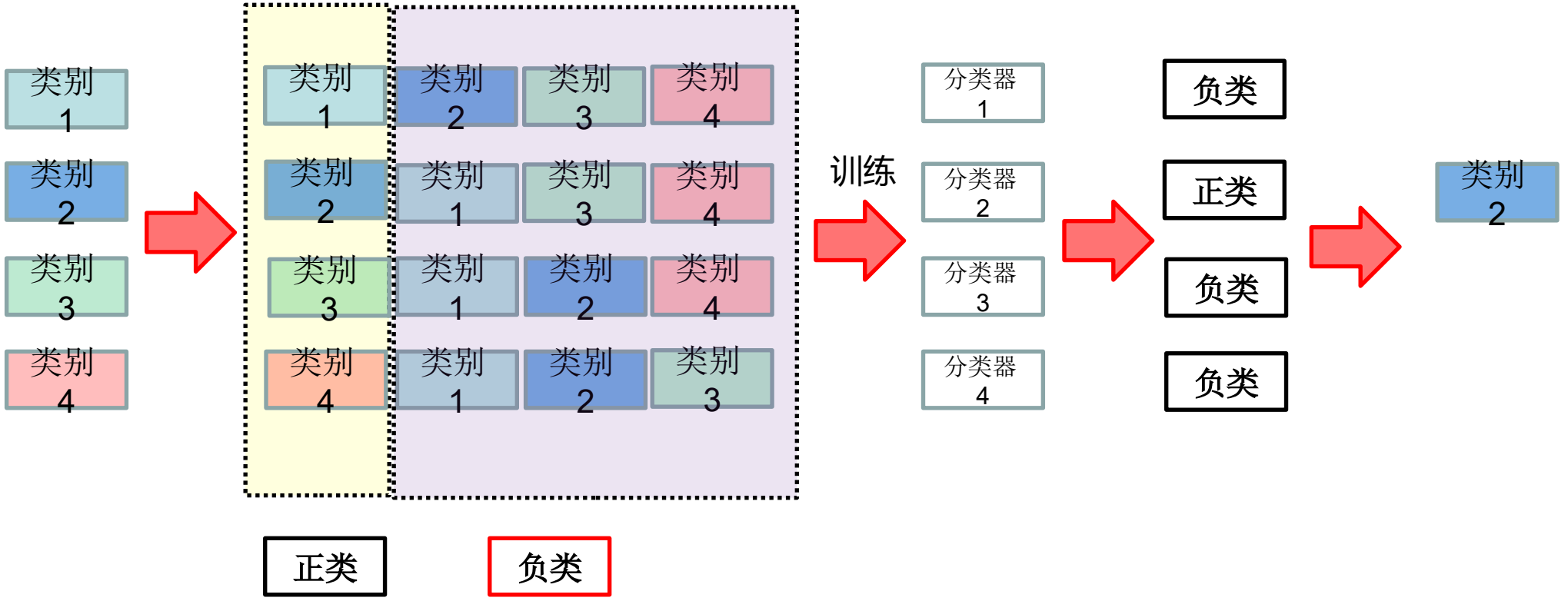
- “一对一”策略（One vs. One）
  - 假定某个多分类问题中有N个类别，我们将这 个类别进行两两配对，就可以将该多分类问题转换成为$\binom{N}{2}$个二分类问题。



测试数据被分到类别 1

1. 前言

- “一对剩余”策略（One vs. Rest）
  - 每次将一个类别作为正类，剩余的所有类别作为负类。



2.kNN 算法

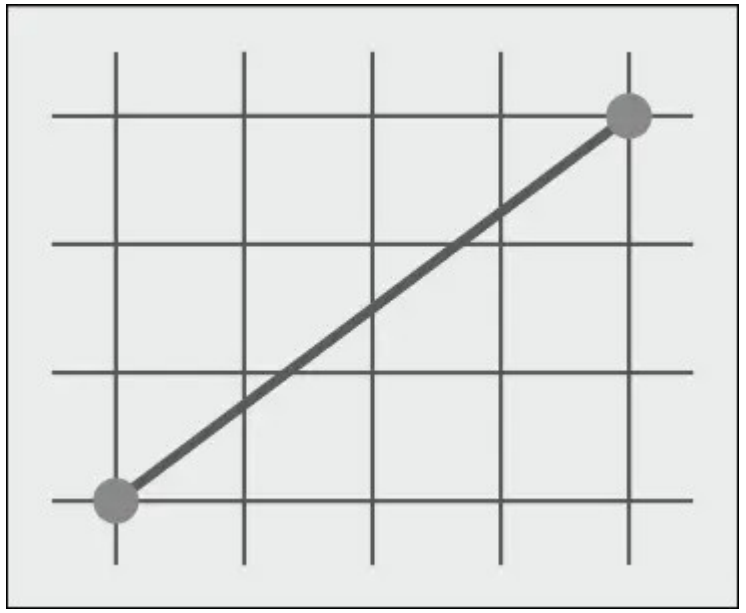
- **k- 最近邻算法**（k-Nearest Neighbors, kNN）是一种机器学习算法中最简单、最基础的有监督分类算法之一。它通过测量不同特征之间的距离确定邻近数据，并使用邻近数据的标签对单个数据点进行预测。既可以用于回归问题，也能用于分类问题。

2.kNN 算法

- **常用的距离指标：**
  - 欧几里得距离：

是一个通常采用的距离定义，指在多维欧几里得空间空间中两个点之间的**真实距离**

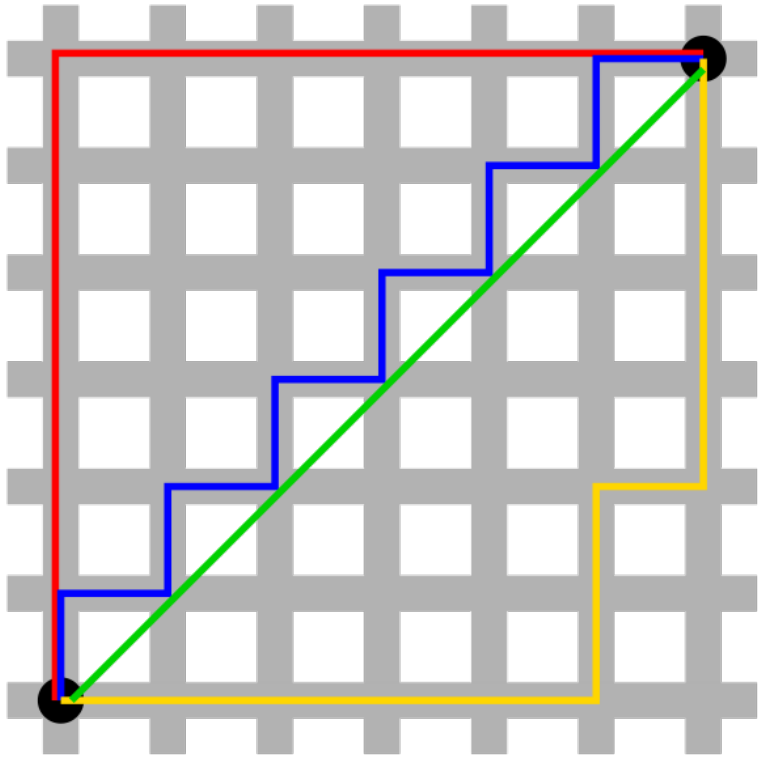
$$d(a,b) = \sqrt{\sum_{k=1}^{n}(x_{1k} - x_{2k})^2}$$



2.kNN 算法

- **常用的距离指标：**
  - 曼哈顿距离：
  - 想象你在城市道路里，要从一个十字路口开车到另外一个十字路口，驾驶距离是两点间的直线距离吗？显然不是，除非你能穿越大楼。实际驾驶距离就是这个“曼哈顿距离”。

$$d(a,b) = \sum_{k=1}^{n} |x_{1k} - x_{2k}|$$



2.kNN 算法

- **常用的距离指标：**
  - 闵可夫斯基距离：
  - 距离度量的一般化形式
- p=2 即为欧氏距离，
- p=1 时则为曼哈顿距离。
- 当 p 取无穷时的极限情况下，可以得到切比雪夫距离

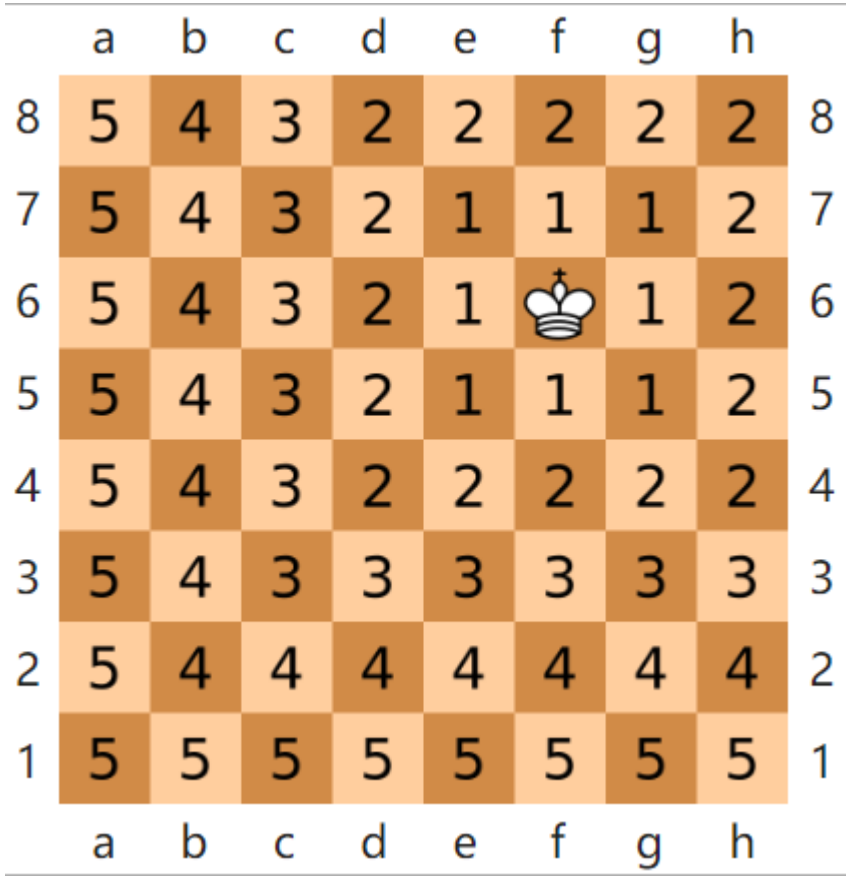
$$d(a,b) = \left(\sum_{k=1}^{n} |x_{1k} - x_{2k}| \right)^{1/p}$$

2.kNN 算法

- **常用的距离指标：**
  - 切比雪夫距离：

- 二个点之间的距离定义是其各坐标数值差绝对值的最大值。
- 国际象棋棋盘上二个位置间的切比雪夫距离是指王要从一个位子移至另一个位子需要走的步数。

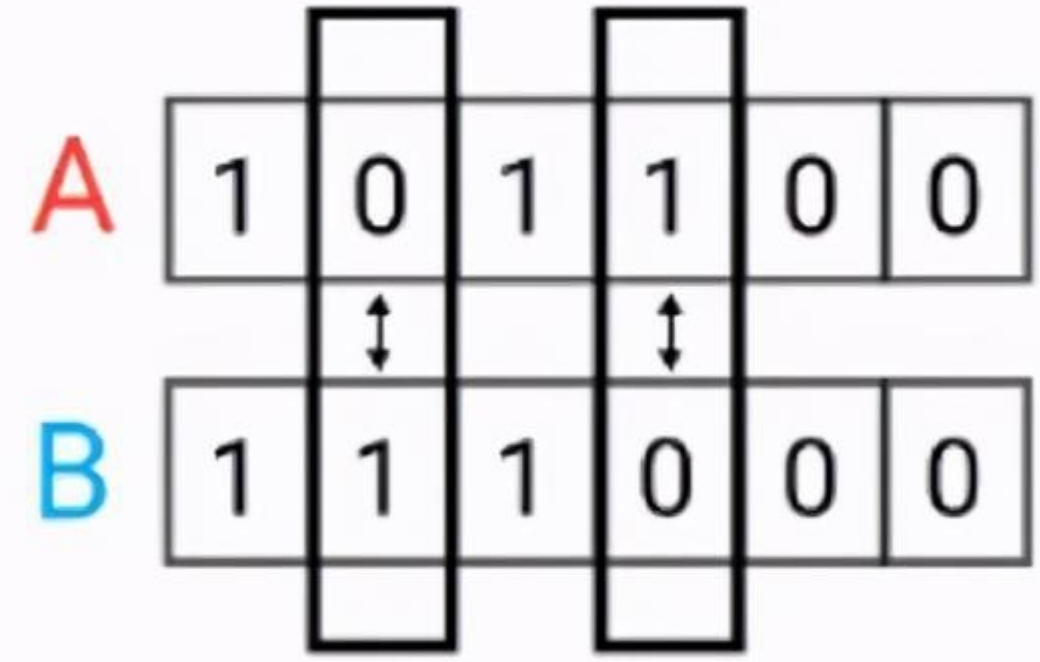
$$d(a,b) = \max_k |x_{1k} - x_{2k}|$$



2.kNN 算法

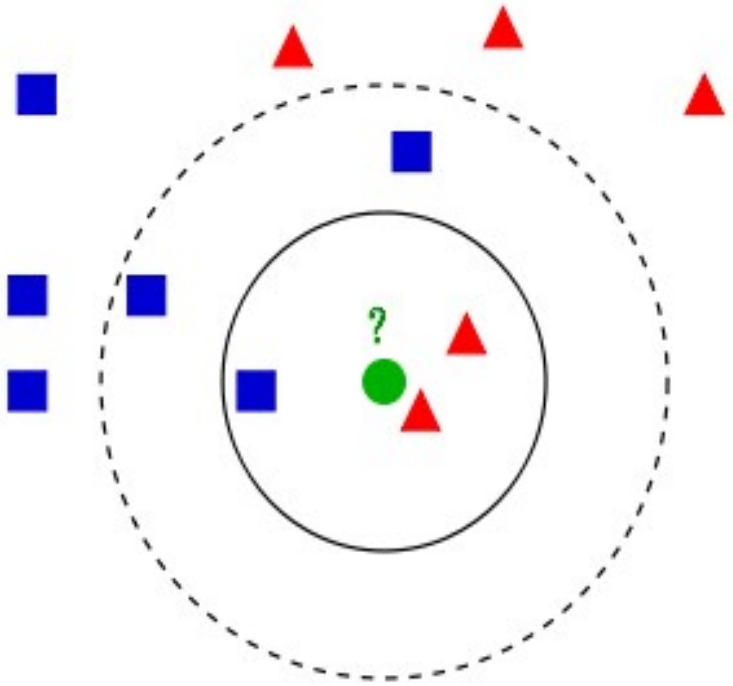
- **常用的距离指标：**
  - 汉明距离：
  - 应用在数据传输差错控制编码中，表示两个（相同长度）字对应位不同的数量。

$$d(a,b) = \frac{1}{n} \sum_{x_{1k} \neq x_{2k}} 1$$



2.kNN 算法

- 算法流程如下：
- 1. 计算测试对象到训练集中每个对象的距离
- 2. 按照距离的远近排序
- 3. 选取与当前测试对象最近的 k 的训练对象，作为该测试对象的邻居
- 4. 统计这 k 个邻居的类别频次
- 5.k 个邻居里频次最高的类别，即为测试对象的类别



2.kNN 算法

- k 的取值：通过**交叉验证**来选择

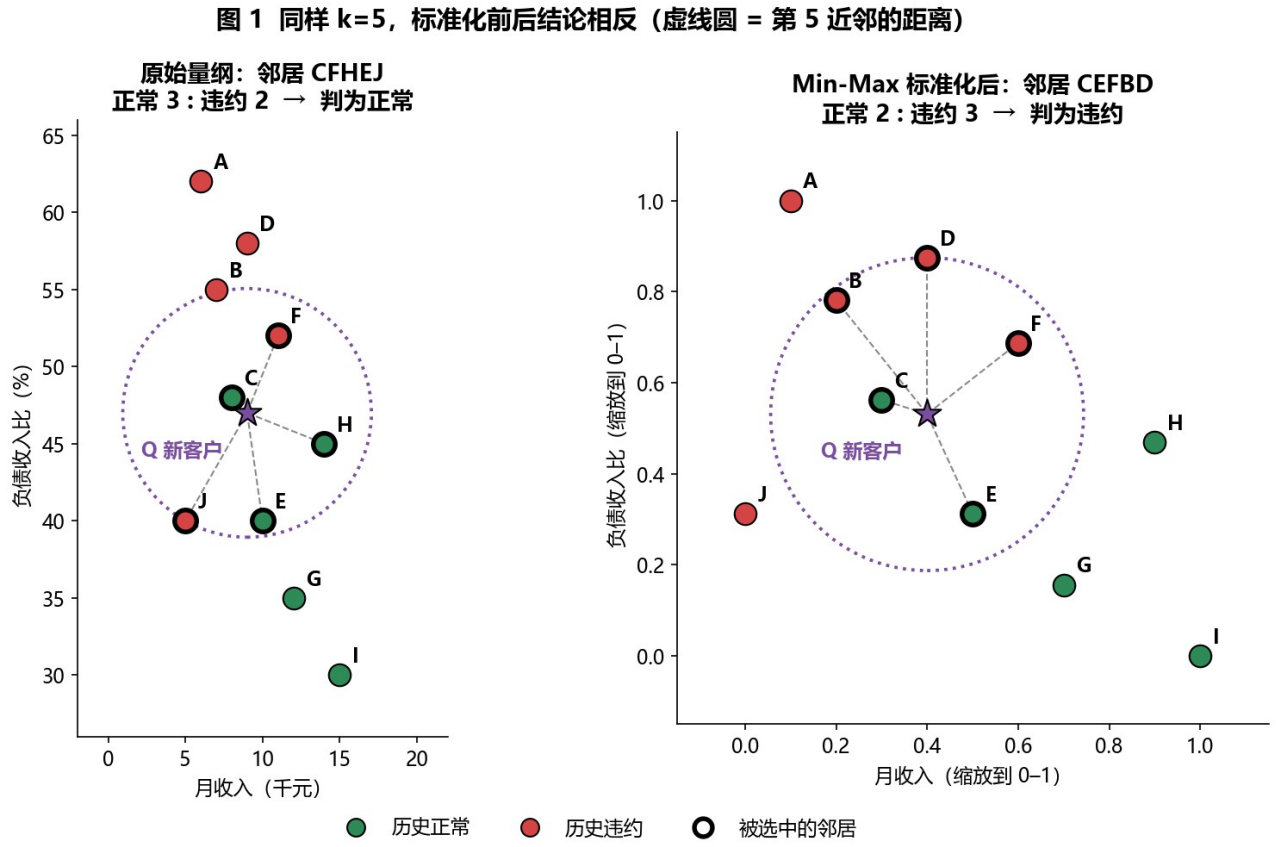
- 首先将样本数据按照一定比例（比如 6:4 ）拆分成训练数据集合验证数据集；从选取一个较小的 k 值开始，不断增加 k 的值，然后计算验证数据集的错误率。最终选择一个合适的 k 值。

2.kNN 算法

- 优点：
  - 简单易用，模型训练时间快；
  - 对异常值不敏感；
  - 随着新样本的增加，算法会根据新数据对模型进行调整，因此适应性比较强；
- 缺点：
  - 对数据规模比较敏感， kNN 本质上是一种惰性算法，与其他分类器相比，它占用了更多的内存。
  - kNN 算法在高位数据输入时表现不佳，由于维数灾难， kNN 容易出现过拟合的情况。利用特征选择和降维技术可以在一定程度上解决这些问题。

尺子变了，邻居就变了

同样的 k=5，标准化前后判决相反。



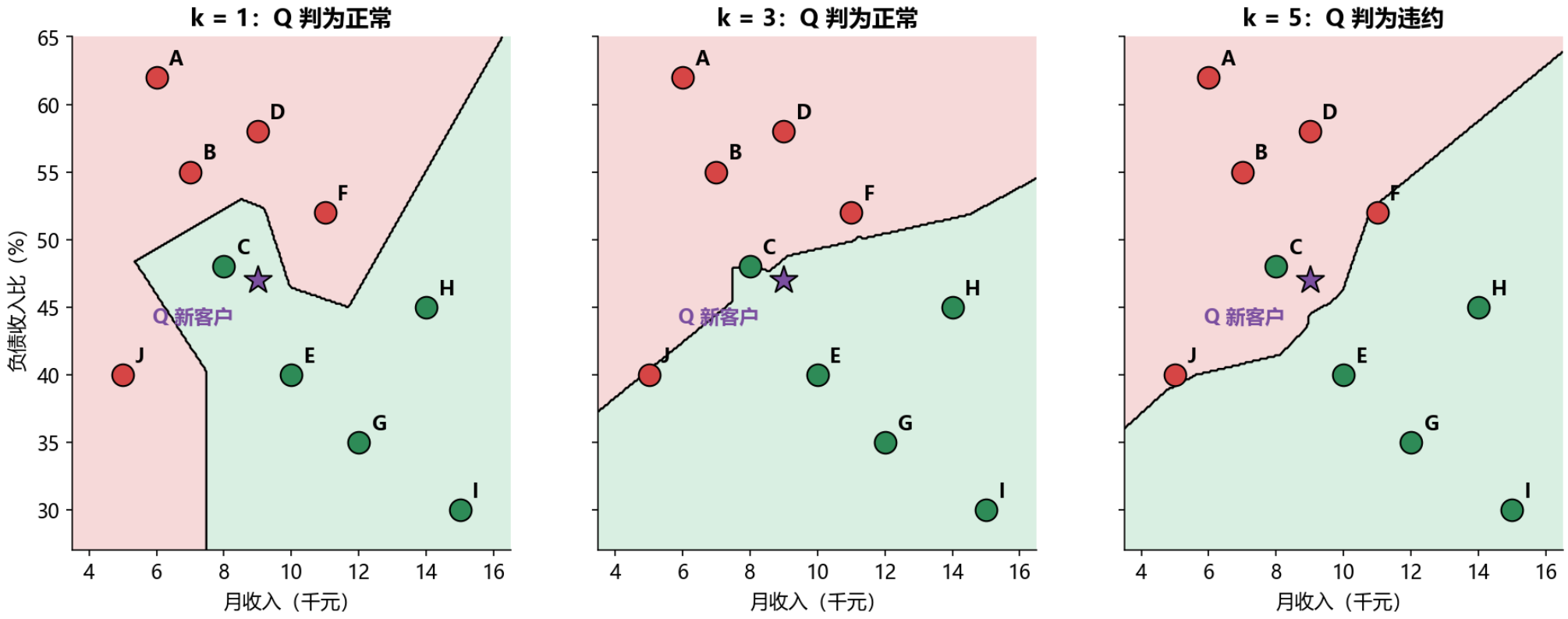
- 原始量纲下，负债收入比的跨度是月收入的 3 倍多，距离主要由它决定。
- 标准化后，k=5 的邻居从 H、J 变为 B、D，判决从“正常”翻为“违约”。
- kNN 没有模型，只有一把尺子；量纲不同的特征必须先标准化。

案例：10 位历史借款人的小额贷款审批（教学构造数据）。

k 值决定边界

k=1 边界贴着每个点； k=5 边界接近一条斜线。

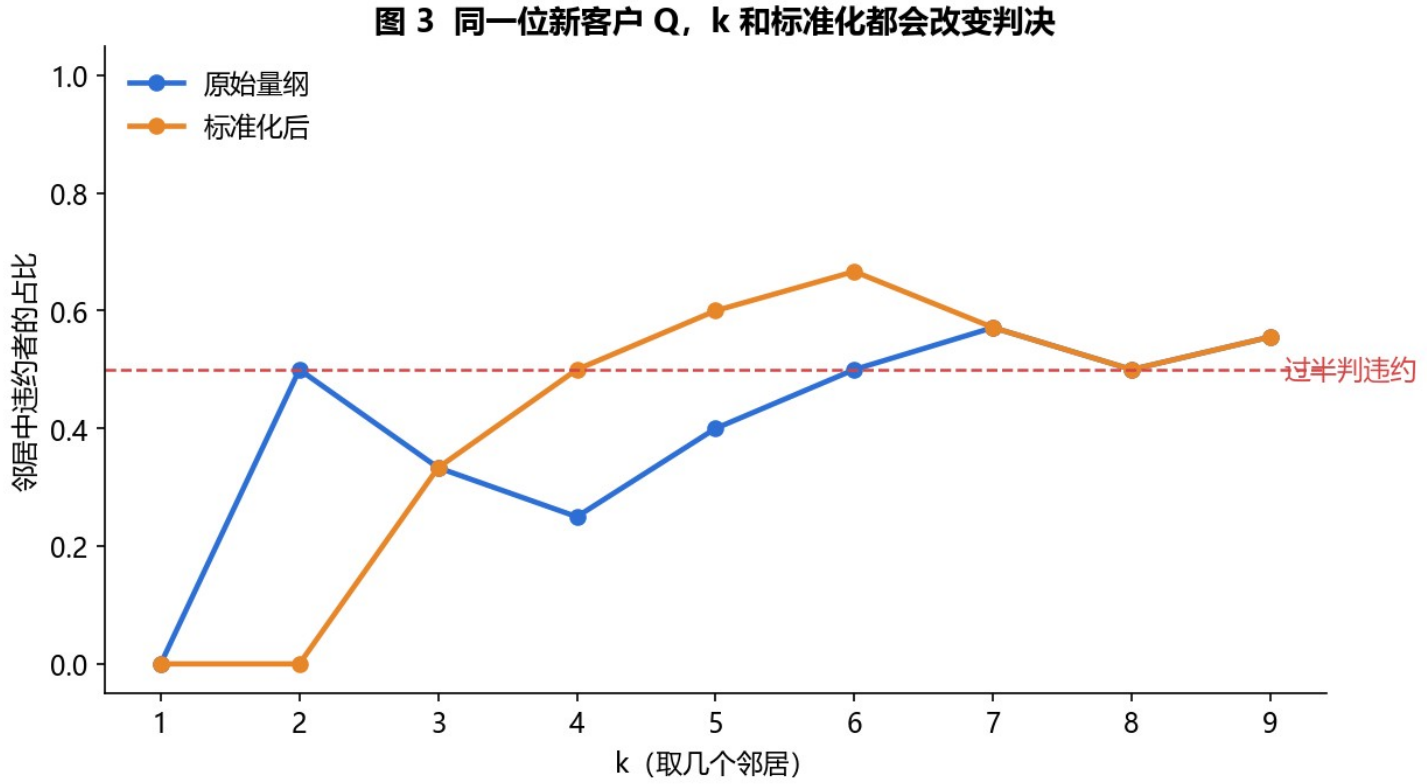
图 2 k 越小边界越碎，k 越大边界越平滑（标准化后计算；绿区判正常，红区判违约）



k 太小像只听一位客户， k 太大像只看全行平均违约率； k 用验证集选。

同一客户，结论随 k 摇摆

纵轴是邻居中违约者的占比；超过 50% 判违约。

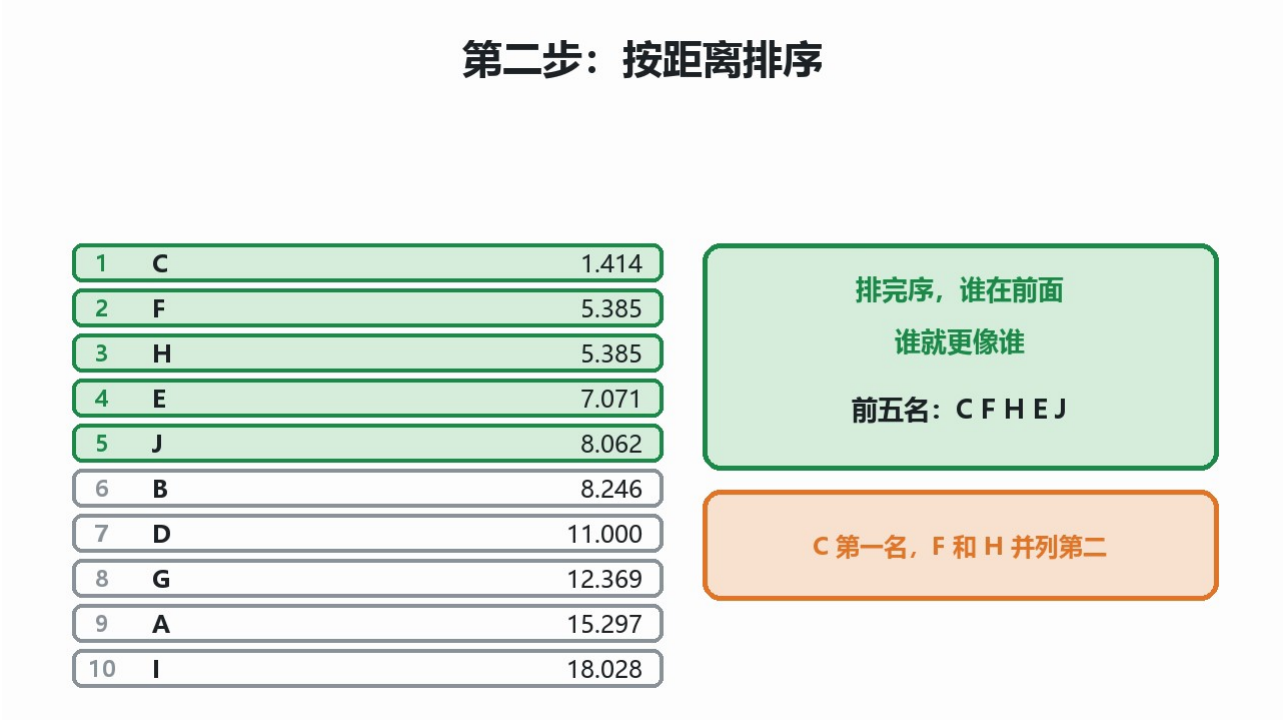


- 原始量纲在 k=5 时低于 50%，标准化后高于 50%。
- 两条线在 k≥7 后重合，因为邻居覆盖了大半样本。
- 边缘客户的判决会随 k 和尺子来回摇，k 必须用验证集选。

蓝线：原始量纲；橙线：标准化后。教学构造数据。

视频： kNN 算法步骤

2 分 48 秒；用同一组贷款数据演示五步流程。



播放视频

五步流程

算距离 → 排序 → 取 k 个邻居 → 统计类别 → 多数投票。

视频重点：标准化前后， k=5 的邻居和判决发生变化。

播放：视频文件位于“视频”子文件夹。

视频来源：本地课件配套视频。

3. 朴素贝叶斯

- **贝叶斯分类器：**是在既定概率框架下进行决策的方法。对于一般的分类任务而言，假定相关的概率全部已知，我们可以利用这些概率来判断最优的标签。
- **先验概率：**即基于统计的概率，是基于以往历史经验和分析得到的结果，不需要依赖当前发生的条件。
- **后验概率：**是从条件概率而来，由因推果，是基于当下发生了事件之后计算的概率，依赖于当前发生的条件。
- **条件概率：**记事件A发生的概率$P(A)$，事件B发生的概率$P(B)$，则在事件B发生的前提下，事件A发生的概率即为条件概率

$$P(A|B)=\frac{P(AB)}{P(B)}$$

3. 朴素贝叶斯

- **贝叶斯公式：** 基于条件概率，通过 P(B|A) 来求取 P(A|B)

$$P(A|B)=\frac{P(B|A)P(A)}{P(B)}$$

  - 将 A 看成是“规律”， B 看成是“现象”，那么贝叶斯公式看成

$$P(\text{规律}\,|\,\text{现象})=\frac{P(\text{现象}|\text{规律})P(\text{规律})}{P(\text{现象})}$$

- **全概率公式：**

$$P(B)=\sum_{i=1}^{n}P(B|A_i)P(A_i)$$

3. 朴素贝叶斯

- **朴素贝叶斯**（Naïve Bayes）是一种经典的分类算法，它的经典案例是用于文本分类。朴素贝叶斯本质上是典型的生成式方法，首先由训练数据学习联合概率分布$P(A,B)$，然后求解后验概率分布$P(B|A)$
- 朴素贝叶斯方法的基本假设是特征条件假设，即假设每个特征之间没有联系

$x = (x_1, \ldots, x_n)$ 数据样本

$c = (c_1, \ldots, c_k)$ 类标签

3. 朴素贝叶斯

- 对于给定的新样本x，判断其属于哪个标记的类别，根据贝叶斯定理，可以得到x属于类别c的概率

$$P(c|x)=\frac{P(x|c)P(c)}{P(x)}$$

- **后验概率最大的类别**记为预测类别

$$\underset{c_k}{\operatorname{argmax}} P(c_k|x)$$

3. 朴素贝叶斯

- 朴素贝叶斯算法对条件概率分布做出**独立性假设**，通俗地讲假设各个维度的特征相互独立
- 条件概率可以转换成为

$$P(x|c_k) = P(x_1, \ldots, x_n|c_k) = \prod_{i=1}^{n} P(x_i|c_k)$$

- 代入贝叶斯公式

$$P(c_k|x) = \frac{P(c_k) \times \prod_{i=1}^{n} P(x_i|c_k)}{\sum_k P(c_k) \times \prod_{i=1}^{n} P(x_i|c_k)}$$

3. 朴素贝叶斯

- **朴素贝叶斯分类器**可以被表示为

$$f(x) = \arg\max_{c_k} P(c_k|x) = \arg\max_{c_k} \frac{P(c_k) \times \prod_{i=1}^{n} P(x_i|c_k)}{\sum_k P(c_k) \times \prod_{i=1}^{n} P(x_i|c_k)}$$

- 对所有的类别，上式中的分母的值是一样的，所以**忽略分母**

$$f(x) = \arg\max_{c_k} P(c_k) \times \prod_{i=1}^{n} P(x_i|c_k)$$

3. 朴素贝叶斯

- 案例：西瓜数据集 - 训练集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

3. 朴素贝叶斯

- 首先评估**先验概率**$P(c)$，可知P(好瓜)=8/17和P(坏瓜)=9/17，然后估计每个属性估计得出**条件概率**$P(x|c)$

$P_{青绿|好瓜} = P(色泽 = 青绿|好瓜) = \frac{3}{8}$　$P_{青绿|坏瓜} = P(色泽 = 青绿|坏瓜) =$　$P_{蜷缩|好瓜} = P(根蒂 = 蜷缩|好瓜) =$

$P_{蜷缩|坏瓜} = P(根蒂 = 蜷缩|坏瓜) =$　$P_{浊响|好瓜} = P(敲声 = 浊响|好瓜) =$　$P_{浊响|坏瓜} = P(敲声 = 浊响|坏瓜) =$

$P_{清晰|好瓜} = P(纹理 = 清晰|好瓜) =$　$P_{清晰|坏瓜} = P(纹理 = 清晰|坏瓜) =$　$P_{凹陷|好瓜} = P(脐部 = 凹陷|好瓜) =$

$P_{凹陷|坏瓜} = P(脐部 = 凹陷|坏瓜) =$　$P_{硬滑|好瓜} = P(触感 = 硬滑|好瓜) =$　$P_{硬滑|坏瓜} = P(触感 = 硬滑|坏瓜) =$

3. 朴素贝叶斯

- 假定每个属性与其他属性无关，即**属性独立。**
- **西瓜数据集 - 测试集**
- 对测试数据计算未归一化分类得分（先验概率 8/17 、 9/17 ），可得
- 朴素贝叶斯分类器可以将测试样本划分为**好瓜**

测试样本 1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	?

$$P(\text{好瓜}) = P_{\text{青绿}|\text{好瓜}} \times P_{\text{蜷缩}|\text{好瓜}} \times P_{\text{浊响}|\text{好瓜}} \times P_{\text{清晰}|\text{好瓜}} \times P_{\text{凹陷}|\text{好瓜}} \times P_{\text{硬滑}|\text{好瓜}} = 0.087$$

$$P(\text{坏瓜}) = P_{\text{青绿}|\text{坏瓜}} \times P_{\text{蜷缩}|\text{坏瓜}} \times P_{\text{浊响}|\text{坏瓜}} \times P_{\text{清晰}|\text{坏瓜}} \times P_{\text{凹陷}|\text{坏瓜}} \times P_{\text{硬滑}|\text{坏瓜}} = 1.62 \times 10^{-3}$$

3. 朴素贝叶斯

- 若某个属性值在训练集中没有与某个类同时出现过，这个时候上述判别方法会出现问题。例如，在使用上述西瓜数据集训练分类器问题时，对于一个“**敲声 = 清脆**”的测试样本时，计算得到概率是

$$P(\text{敲声=清脆}|\text{好瓜}) = P(\text{敲声=清脆}|\text{坏瓜}) = 0$$

- 有了上述的值，不论该样本的其他属性是什么，哪怕其他的属性上明显是好瓜，分类的结果都会是“坏瓜”，**这显然有问题**。

3. 朴素贝叶斯

- **拉普拉斯平滑：**
- 为了解决零概率的问题，法国数学家拉普拉斯最早提出用**加 1** 的方法估计没有出现过的现象的概率

$$P(x_i|D_c) = \frac{|D_{c,x_i}| + 1}{|D_c| + N_i}$$

$\|D_{c,x_i}\|$	$\|D_c\|$	N_i
类别c属性i的样本数	类别c的样本数	属性i可能的取值

3. 朴素贝叶斯

- **拉普拉斯平滑案例：西瓜数据集**
- 计算**类先验概率**

$$\hat{P}(\text{好瓜}) = \frac{8+1}{17+2} \approx 0.474 \qquad \hat{P}(\text{坏瓜}) = \frac{9+1}{17+2} \approx 0.526$$

- 计算**条件概率**

$$\hat{P}_{\text{青绿|好瓜}} = \hat{P}(\text{色泽} = \text{青绿|好瓜}) = \frac{3+1}{8+3} \approx 0.364 \quad \hat{P}_{\text{青绿|坏瓜}} = \hat{P}(\text{色泽} = \text{青绿|坏瓜}) = \frac{3+1}{9+3} \approx 0.333$$

$$\hat{P}_{\text{清脆|好瓜}} = \hat{P}(\text{敲声} = \text{清脆|好瓜}) = \frac{0+1}{8+3} \approx 0.091$$

3. 朴素贝叶斯

- **优点：**
  - 朴素贝叶斯模型有稳定的分类效率。
  - 对小规模的数据表现很好，能处理多分类任务，适合于增量训练，尤其是数据量超出内存时，可以一批批去增量训练。
  - 对缺失数据不太敏感，算法也比较简单，常用于文本分类。
- **缺点：**
  - 需要知道先验概率，且先验概率很多时候取决于假设，假设的模型可以有很多种，因此在某些时候会由于假设的先验模型的原因导致预测效果不佳
  - 对输入数据的表达形式很敏感（离散、连续，值极大极小之类的）。

半朴素贝叶斯分类器

- 以适当考虑一部分属性之间的相互依赖信息，从而既不需要进行完全联合概率计算，又不至于彻底忽略了比较强的相互依赖关系，“独依赖估计”是半朴素贝叶斯分类器中最常用的一种策略。顾名思义，所谓“独依赖”，就是**假设每个属性在类别之外最多依赖于一个其他属性**。

- 对每个属性而言，若每个父属性已知的情况下，则可以采用类似估计条件概率的方法来估计条件概率

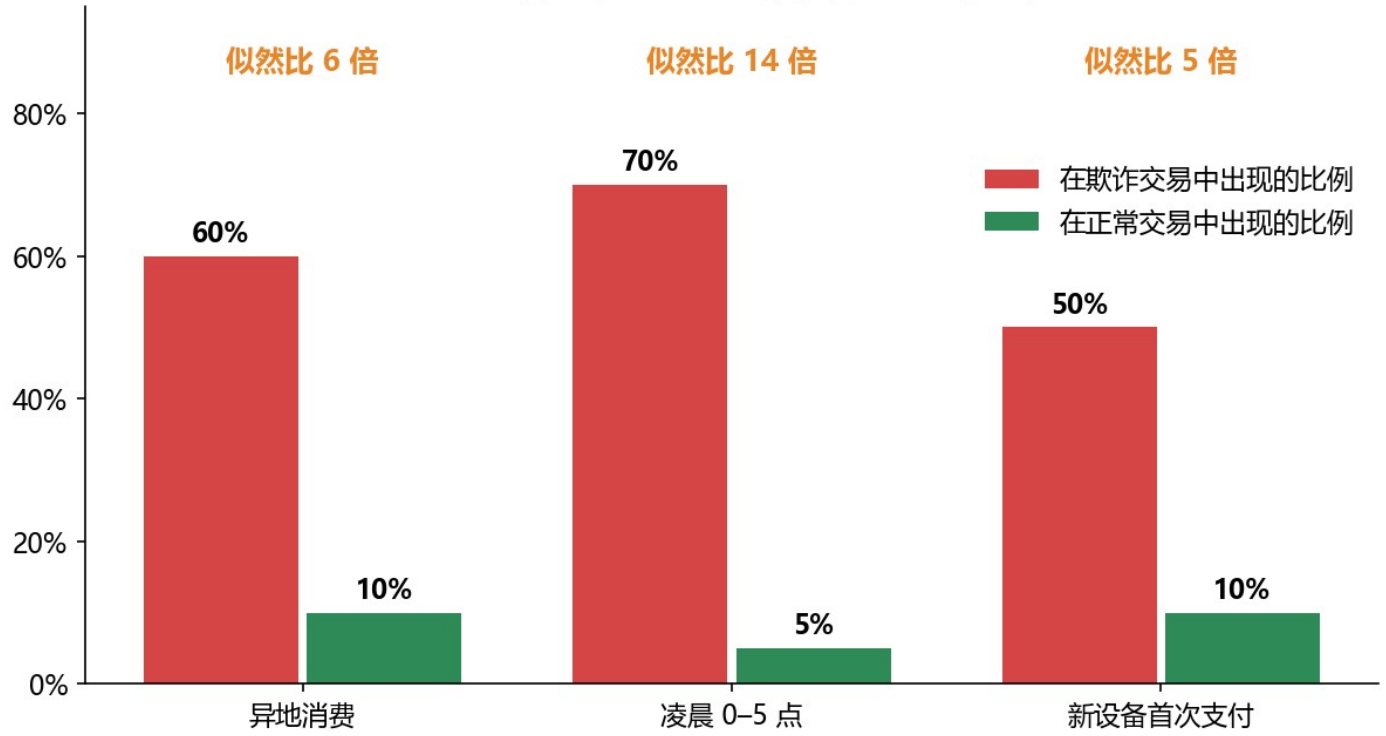
$$P(c|x) \propto P(c) \prod_{i=1}^{d} P(x_i|c, pa_i)$$

父属性

一条证据有多强

看它在两类里差得多不多，而不是它本身出现得多不多。

图 4 似然：每条证据在两类交易里各有多常见



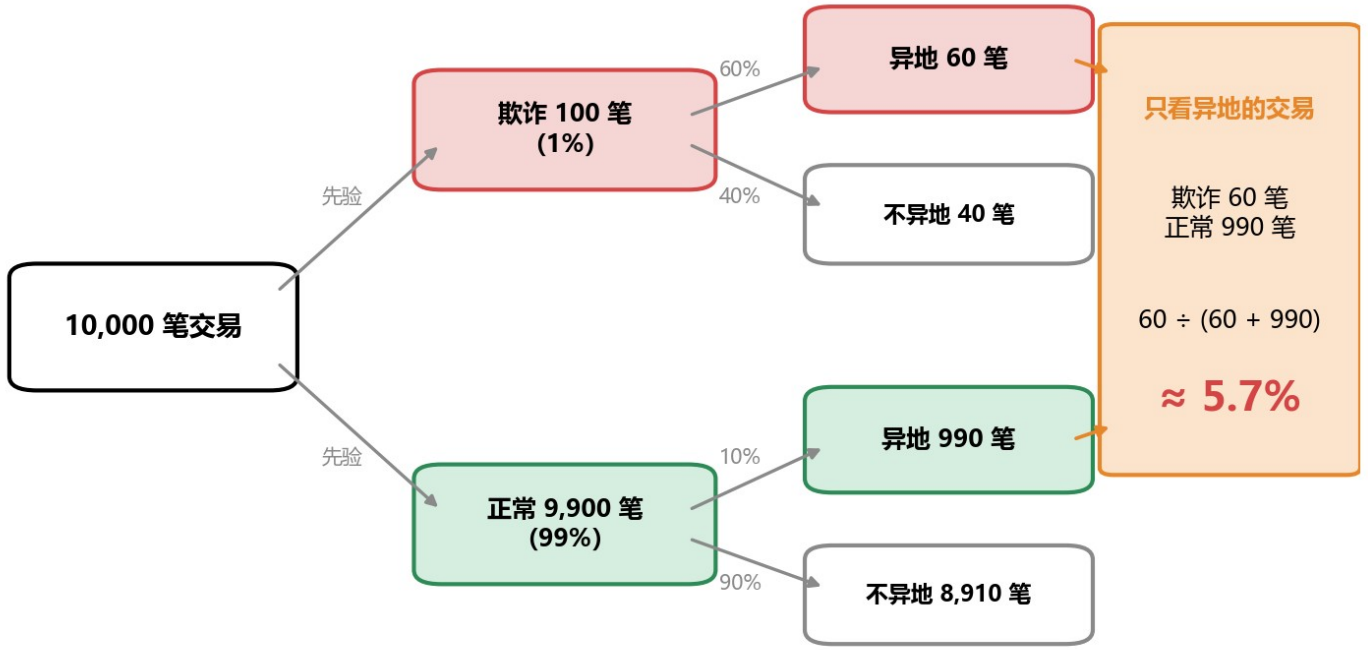
- 似然比 = 欺诈交易中出现比例 ÷ 正常交易中出现比例。
- “凌晨”的似然比约 14 倍，是最强的一条；异地约 6 倍，新设备约 5 倍。
- 单一“异地”信号只能把欺诈概率从 1% 推到 5.7%，先验很关键。

案例：10,000 笔信用卡交易，100 笔欺诈（教学构造数据）。

数人头就能算贝叶斯

把概率换成一万笔交易里有多少笔，公式就变成数数题。

图 5 用「频数树」看贝叶斯公式：异地交易里，正常交易的数量是欺诈的 16 倍以上



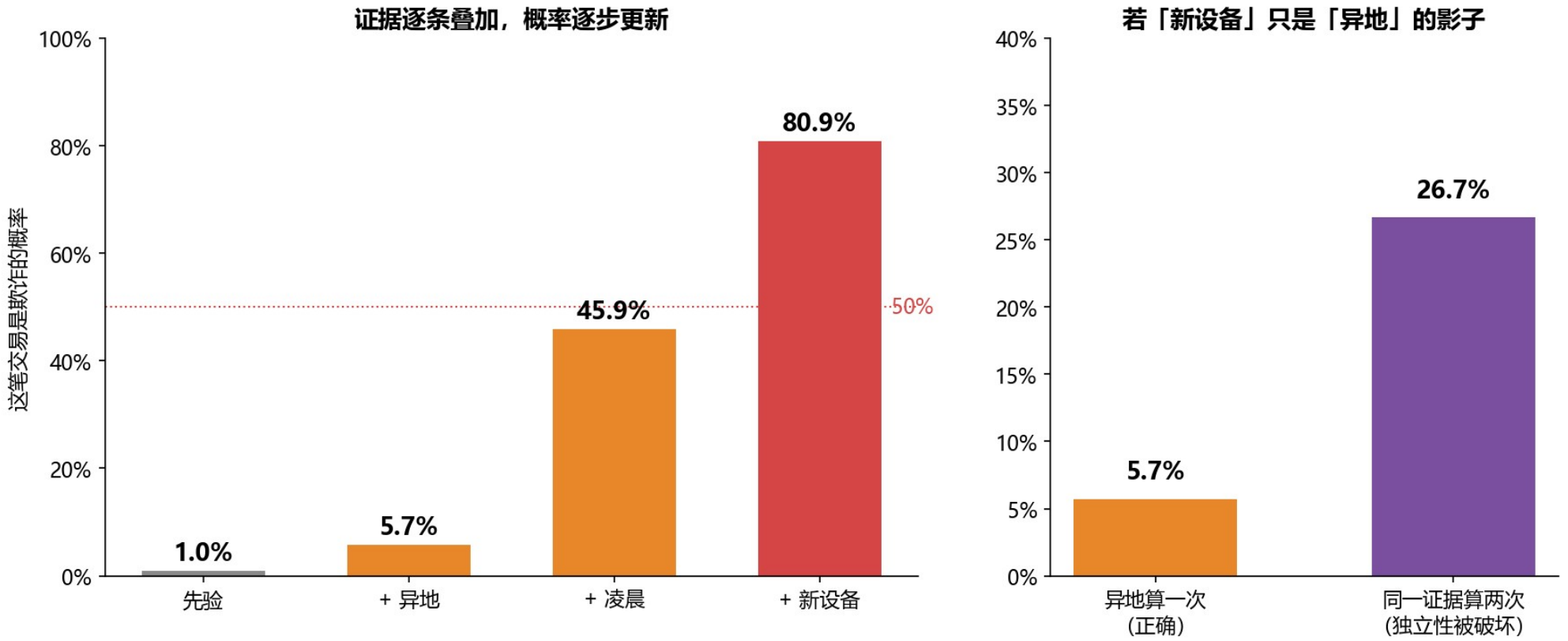
- 异地交易共 1,050 笔，其中欺诈 60 笔。
- 60 ÷ 1,050 ≈ 5.7%，与贝叶斯公式结果一致。
- 先验是第一层分叉，似然是第二层分叉，后验是把同一条件下的两类放在一起比。

频数树适合先建立直觉，再回到公式。

三条证据叠加： 1% → 80.9%

每加一条证据，就把上一步的后验当作新的先验。

图 6 左：三条证据把 1% 推到约 81%；右：相关证据被重复计算，概率被放大

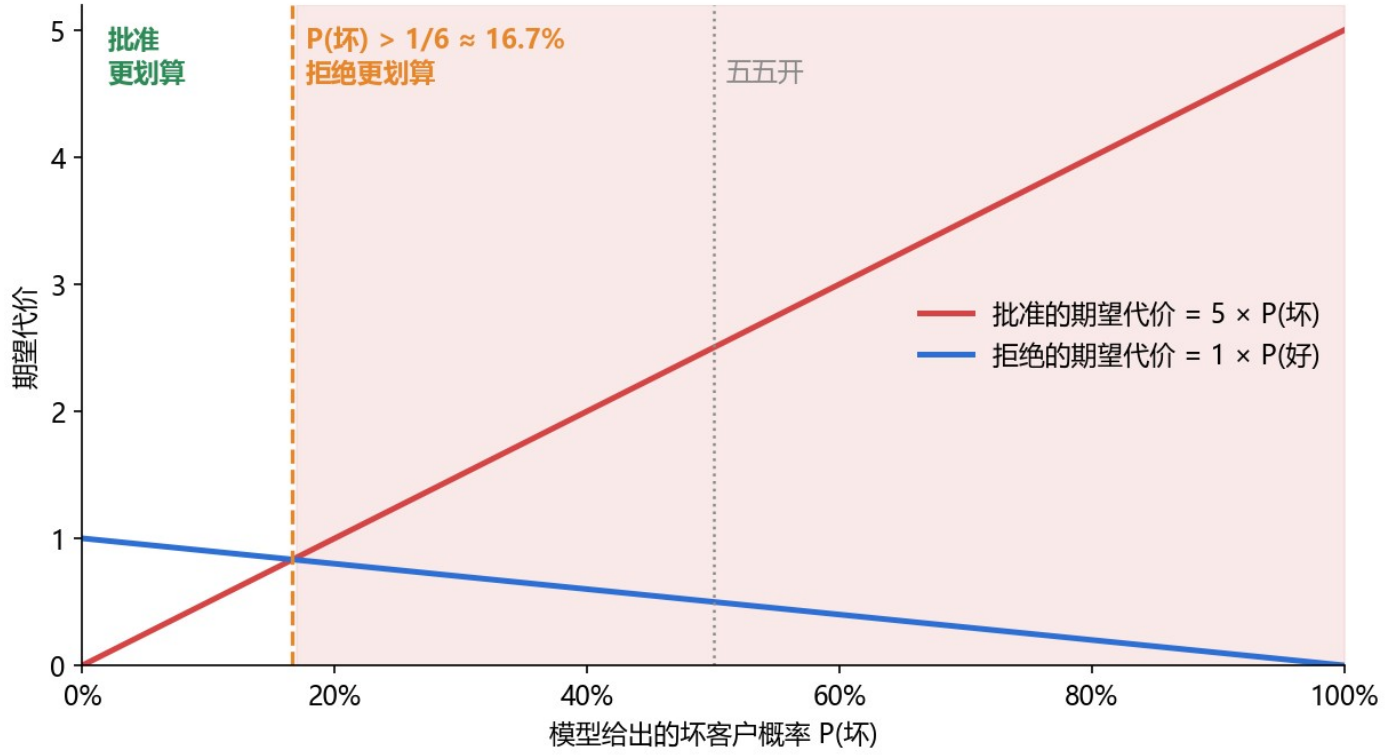


右图：当“新设备”只是“异地”的影子，独立性假设被破坏，概率被重复放大。

误判代价决定阈值

坏客户判好客户的代价是 5，好客户判坏客户的代价是 1。

图 7 误判代价决定阈值：坏判好代价 5，好判坏代价 1



- 当 5 × P(坏) > 1 × P(好)，即 P(坏) > 16.7% 时，拒绝更便宜。
- 模型只负责给出概率，阈值由业务代价决定。
- 错放一个坏人的代价越大，拒贷线越往左移。

来源：UCI Statlog German Credit；代价矩阵见 german.doc。

视频：朴素贝叶斯算法

3 分 16 秒；三条证据如何叠加。



播放视频

视频主线

先验 × 似然 → 后验；三条证据：异地、凌晨、新设备。

视频重点：独立性假设何时失效，概率为何被重复放大。

播放：视频文件位于“视频”子文件夹。

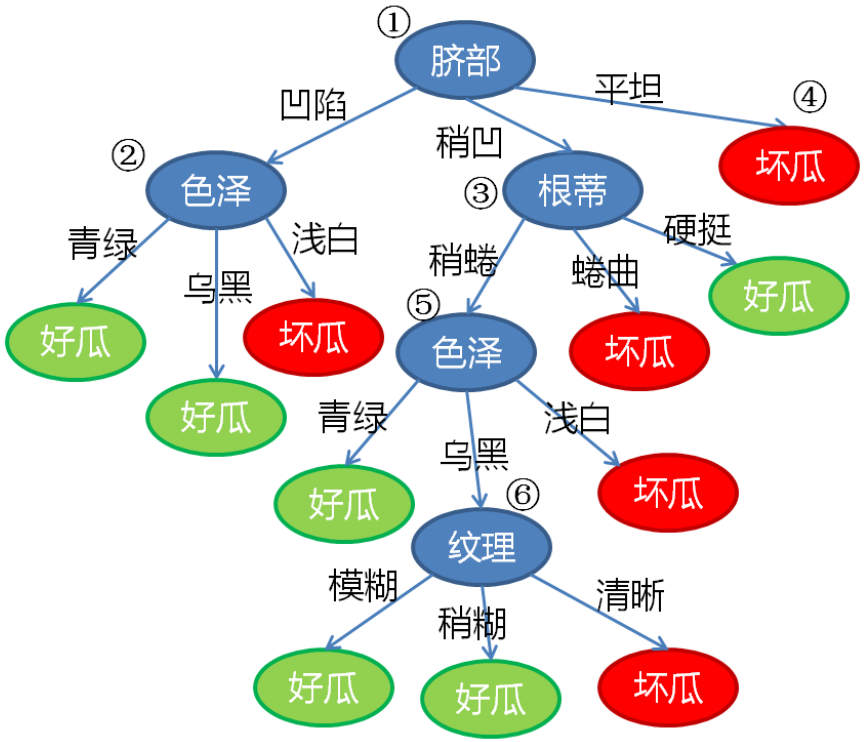
视频来源：本地课件配套视频。

4. 决策树

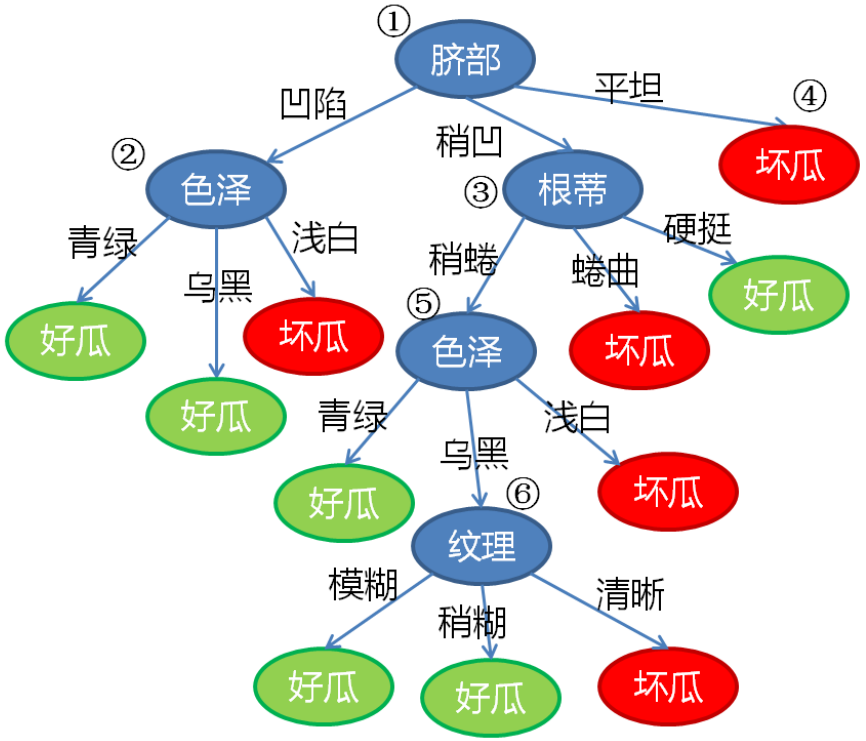
- 决策树：从训练数据中学习得出一个树状结构的模型。
- 决策树属于判别模型。
- 决策树是一种树状结构，通过做出一系列决策（选择）来对数据进行划分，这类似于针对一系列问题进行选择。
- 决策树的决策过程就是从根节点开始，测试待分类项中对应的特征属性，并按照其值选择输出分支，直到叶子节点，将叶子节点的存放的类别作为决策结果。

4. 决策树

- 以西瓜的分类为例，我们在对“这个瓜是否是好瓜”这个问题进行判断时，通常需要经过一系列的分析，首先我们要看“这个瓜是什么颜色？”，如果是“青绿色”，我们再看“这个瓜的根蒂是什么状态？”，如果是“蜷缩”，我们再来判断“这个瓜敲起来是什么声音？”，如果是“浑浊”。最后，基于上述分析，我们可以得到最后的决策：这是一个**好瓜**。



4. 决策树



- 决策树算法是一种归纳分类算法，它通过对训练集的学习，挖掘出有用的规则，用于对新数据进行预测。
- 决策树算法属于监督学习方法。
- 决策树归纳的基本算法是贪心算法，自顶向下来构建决策树。
- 贪心算法：在每一步选择中都采取在当前状态下最好 / 优的选择。
- 在决策树的生成过程中，分割方法即属性选择的度量是关键。

4. 决策树

- **决策树的主要步骤如下：**
- **输入：** 训练数据集D，属性集A和阈值ε。
- **输出：** 决策树T

- （1）如果训练集D中所有实例都属于用一个类别D_k（D_k代表训练样本集D中属于第k类的子集），则设置决策树T为单节点树，并将D_k作为该节点的类别。
- （2）如果属性集A为空集，则设置决策树T为单节点树，并将训练集D中最多的类别作为该节点的类别。如果属性集A不是空集，则根据相关公式评估属性集A中各个特征在决策时的得分，并选择最合适的特征a。
- （3）如果特征a的得分小于阈值ε，则设置决策树T为单节点树，并将D_k作为该节点的类别。

  如果特征a的得分大于阈值ε，则对数据集D进行划分，并重复上述步骤递归生成决策树。

ID3 算法

- ID3 算法就是使用**信息增益**作为准则来选择划分属性。
- 主要步骤如下：
- （1）初始化数据集和特征集。
- （2）计算数据集的信息熵和各个特征的条件熵，并基于此计算得到信息增益的值。
- （3）选择信息增益最大的特征作为决策节点进行划分。
- （4）更新数据集和特征集，即删除前一步使用过的特征，按照特征值进行划分，得到新的数据集。
- （5）对每个子集重复（2）、（3）、（4）步；若子集中的样本已全部属于同一类，或已没有可用特征，则将其作为叶节点，类别取多数类。

ID3 算法

- **信息熵**是一个衡量样本集合纯度最常用的指标

$$Ent(D) = -\sum_{k=1}^{|Y|} p_k \log_2 p_k$$

$p_k(k = 1, \ldots, |Y|)$样本集中第k类的比例

- 假定离散属性a有V个可能的取值,则会产生V个分支。
- 第v个分支节点D中所有属性有特定取值的样本D^v
- 计算出信息熵，并考虑D^v占所有数据集的比例，可以得到信息增益

$$Gain(D, a) = Ent(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} Ent(D^v)$$

ID3 算法

- 信息增益越大，则意味着使用相应属性进行划分所得的信息纯度提升越大。
- 因此我们可以利用信息增益来进行决策树进行划分时候的属性选择

$$a^* = \arg\max_{a \in A} Gain(D, a)$$

ID3 算法—案例

西瓜数据集 - 训练集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

验证集

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否

ID3 算法—案例

- 按照色泽进行分类

色泽	数量	好瓜	坏瓜
青绿	4	2	2
乌黑	4	3	1
浅白	2	0	2

信息熵

$$Ent(D) = -\sum_{k=1}^{|Y|} p_k \log_2 p_k = -\frac{5}{10}\log_2\frac{5}{10} - \frac{5}{10}\log_2\frac{5}{10} = 1$$

条件熵

$$Ent(D|a^{青绿}) = -\frac{2}{4}\log_2\frac{2}{4} - \frac{2}{4}\log_2\frac{2}{4} = 1$$

$$Ent(D|a^{乌黑}) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} = 0.8113$$

$$Ent(D|a^{浅白}) = -\frac{2}{2}\log_2\frac{2}{2} = 0$$

$$Ent(D|色泽) = \frac{4}{10}\times 1 + \frac{4}{10}\times 0.8113 + \frac{2}{10}\times 0 = 0.7245$$

信息增益 $Gain(D, a) = 1 - 0.7245 = 0.2755$

ID3 算法

缺点：

- ID3 算法没有剪枝策略，容易出现过拟合。
- 信息增益会对可取值数量较多的特征在选择上有偏好，这种偏好可能会为算法性能带来不良影响。
- 只能用于处理离散特征。
- 算法没有将缺失属性考虑在内。

C4.5 算法

- C4.5 算法的流程大体上跟 ID3 算法类似，区别主要在于以下几点：
- 1. 利用信息增益准则一般对可取值数量较多的属性有偏好，为了减少这种偏好带来的不良影响， C4.5 算法改用**增益率**来选择划分属性

$$Gain_ratio(D,a) = \frac{Gain(D,a)}{IV(a)}$$

$$IV(a) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

- C4.5 算法还需要在决策树生成过程中进行**剪枝**处理，即通过后剪枝的策略消除过拟合的风险
- C4.5 算法通过对连续属性值进行离散化改造使其能用于生成决策树

C4.5 算法

- 使用表 1 的数据对色泽的增益率进行计算

$$IV(a) = -\frac{4}{10}\log_2\frac{4}{10} - \frac{4}{10}\log_2\frac{4}{10} - \frac{2}{10}\log_2\frac{2}{10} = 1.5219$$

$$Gain_ratio(D, \text{色泽}) = \frac{0.2755}{1.5219} = 0.181$$

- 可以计算得到“色泽”属性增益率的值
  - 信息增益率一般情况下对**取值较少的特征有偏好**。因此， C4.5 并不是直接用增益率最大的特征进行划分，而是采用启发式算法的思想，首先选择一个信息增益高于平均值的特征，再从中选择增益率最高的特征。

C4.5 算法

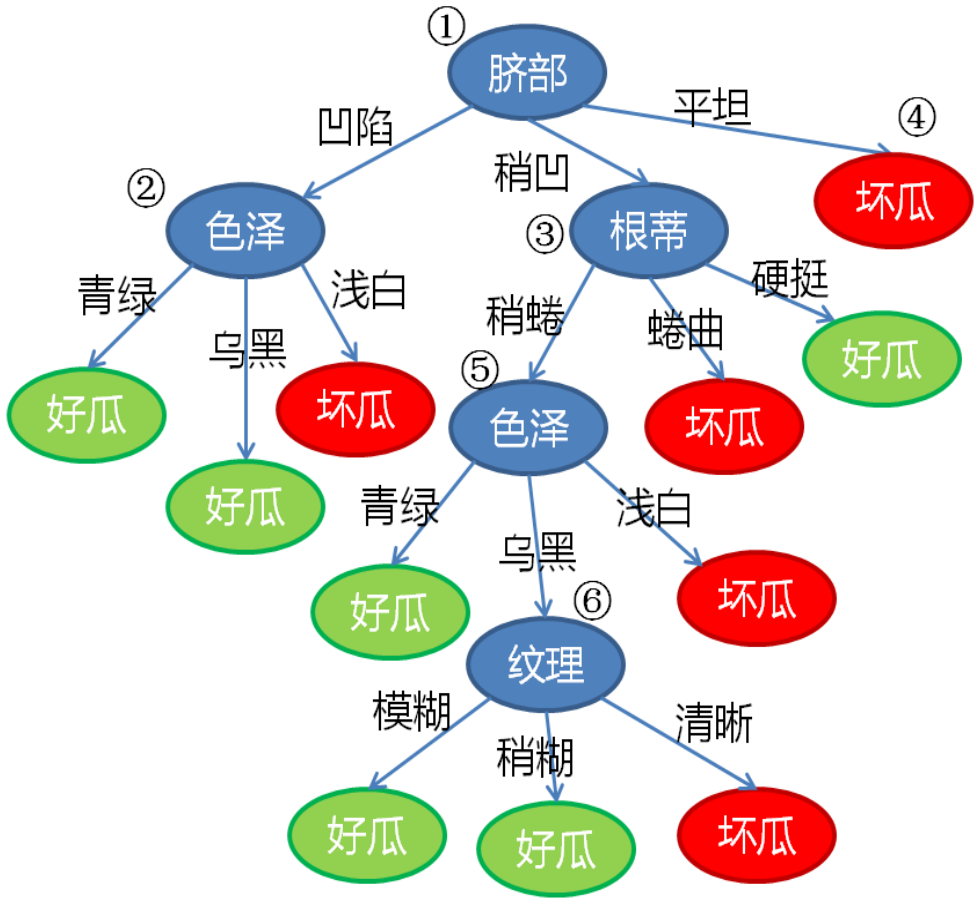
- **剪枝处理**
- 决策树虽然有算法简单、可解释性强的优点，但是常常会遇到“过拟合”的问题，即为了尽可能正确对样本进行分类，我们不断重复划分节点的过程，有些时候会造成决策树分支过多，此时，会造成把训练集样本的一些特点当作是全部数据都具有的一般性质，从而导致过拟合。为了解决上述问题，我们需要主动剪掉一些不必要的分支来减小过拟合的风险。剪枝有预剪枝和后剪枝两种策略。

预剪枝

- 预剪枝需要在决策树生成的过程中，对各个节点在划分之前进行估计，若当前的划分不能带来决策树泛化能力的提升，就不再对当前节点进行划分为叶节点。
- 预剪枝采用的策略有以下三种：
  - 节点内数据样本数低于阈值；
  - 所有特征都已用于划分；
  - 划分后验证集准确率没有提高。

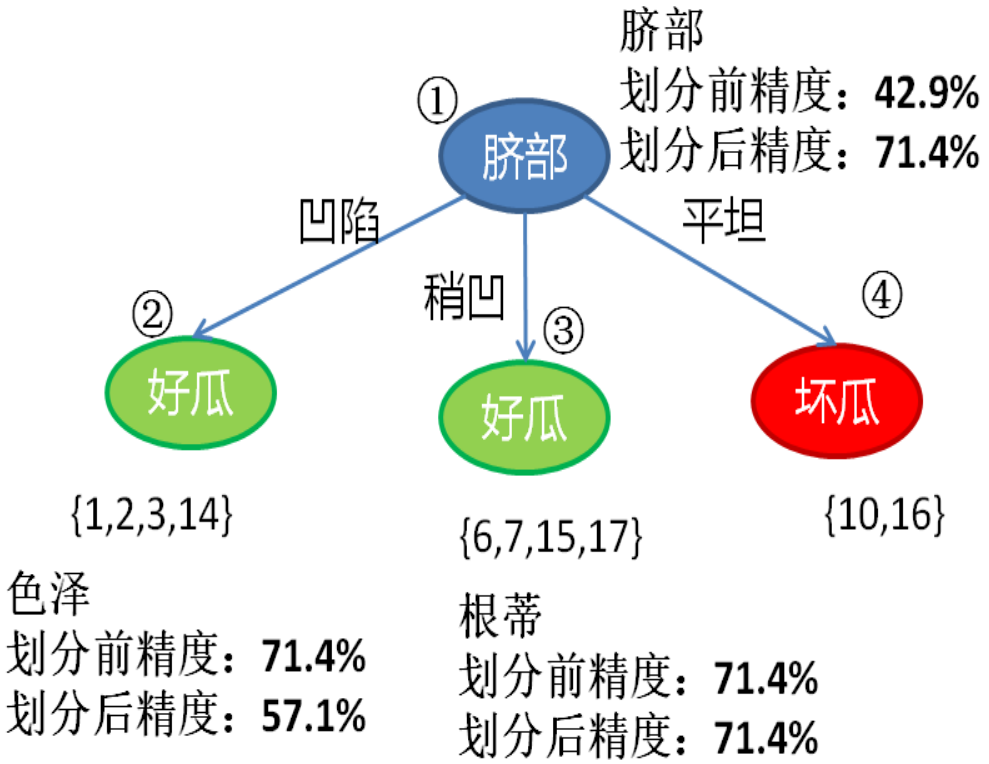
预剪枝

- 在划分之前，所有的训练数据集全部集中在根节点。如果这个时候不进行划分，根据算法，该节点可以被标注为叶节点，类别标记为训练样本数量最多的类别。假定这个叶节点标注为好瓜
- 利用验证集对这个单点决策树进行评估，那么编号 {4,5,8} 的样本可以被正确分类，剩下的几个样本被错分。基于上述结果，验证集的准确率为 3/7=42.8% 。



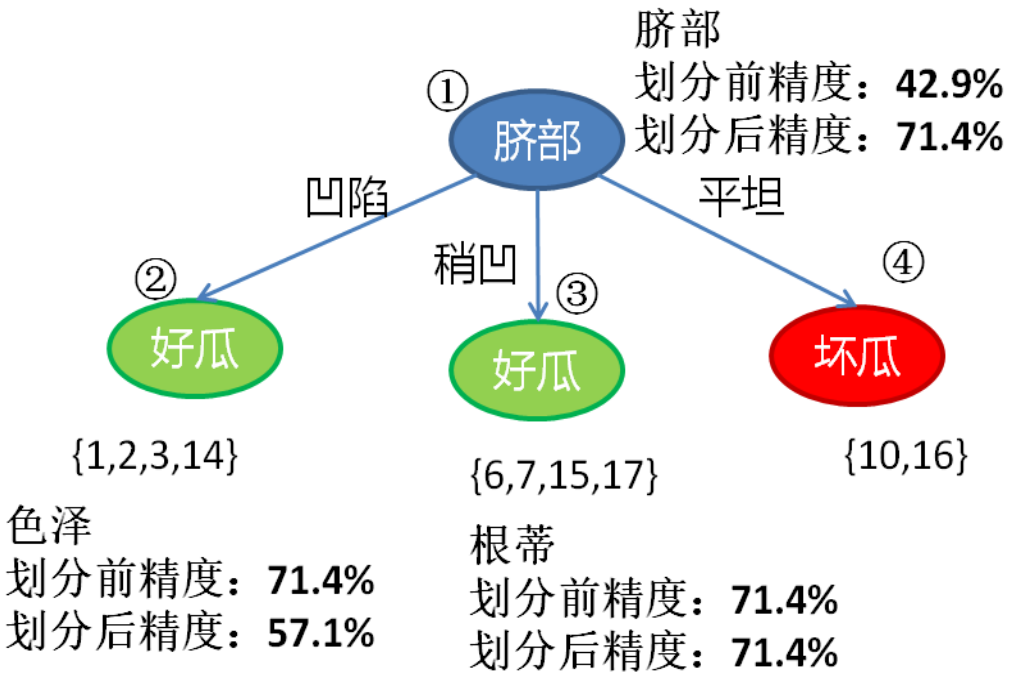
预剪枝

- 在使用“脐部”属性进行划分之后，下图②、③和④三个节点分别被标注为“好瓜”、“好瓜”和“坏瓜”。按照上述的标注，我们可以看到验证集中 {4,5,8,11,12} 的样本被正确分类，此时，验证集的准确率是 5/7=71.4% 。由此，我们可以得出结论，即基于“脐部”属性进行划分能提高决策树的泛化性能。
- 我们对节点②进行进一步的划分，同样基于信息增益原则选择“色泽”属性进行划分，从中可以看到青绿和乌黑的样本被划分成正例，浅白的样本被划分成为负例。然而，我们使用验证集对上述划分进行准确率的验证，我们发现划分前验证集的准确率是 71.4% ，经过了划分，验证集的准确率下降到了 57.1% 。因此，基于预剪枝策略，不再进行这次划分。



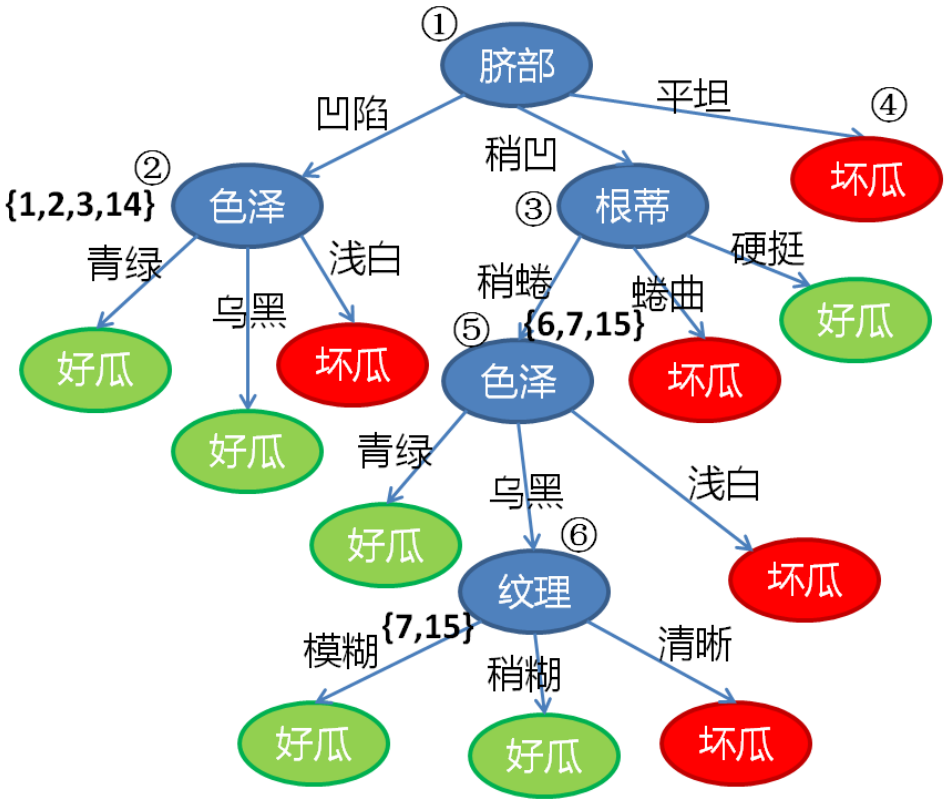
预剪枝

- 在节点③使用“根蒂”属性进行进一步划分，然后利用验证集对划分前后的精度进行了评估，发现划分前后验证集的分类精度不变。因此，上述划分从提升泛化性能的角度来看，显得毫无意义。因此，我们也禁止对上述属性进行划分。
- 对于节点④，我们看到隶属于该节点的所有样本都已经属于一个类别，因此无需进行进一步地划分。



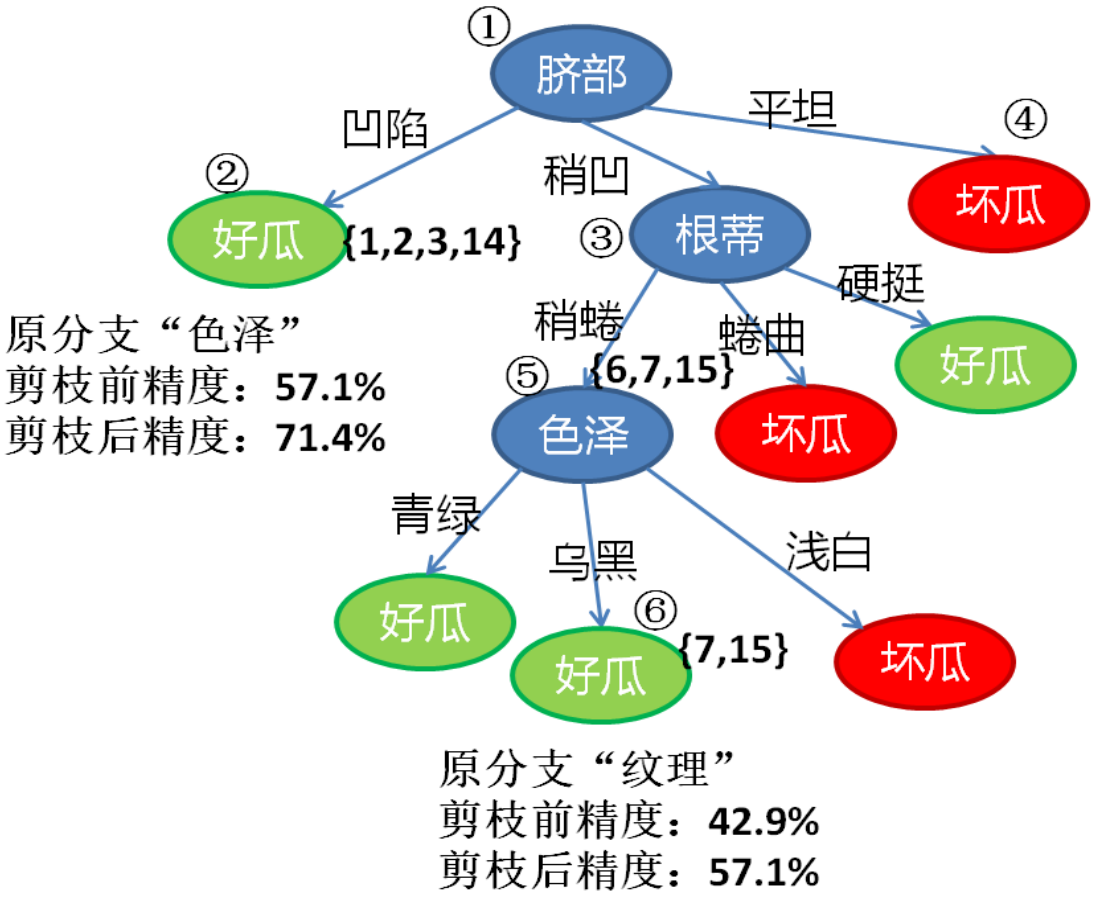
后剪枝

- 后剪枝则是从训练集中生成一棵完整的决策树，然后自下而上对非叶节点进行逐个评估，若将该节点对应的子树替换成叶节点能提高决策树的泛化性能，就将该子树替换成叶节点。
- 首先利用信息增益训练得到一棵完整的决策树，如下图所示。此时，我们利用验证集对决策树进行验证，并得到验证集的准确率为 42.9% 。



后剪枝

- 基于后剪枝策略，我们首先考察节点⑥，将其包含的分支剪掉。此时，我们将节点⑥替换成为一个子叶节点，里面包含 {7,15} 两个训练样本，因此可以被标注为“好瓜”。再利用验证集进行验证可得剪枝之后的精度提升到了 57.1% 。因此，可以对节点⑥进行剪枝。
- 考察节点⑤，将其替换为子叶节点，此时，利用验证集进行验证，发现剪枝之后的精度为 57.1% ，没有提升，因此，不再对节点⑤进行剪枝。



CART 算法

- CART 算法生成的决策树是结构比较简单的二叉树，即把当前样本集划分成两个子样本集，由此生成的每个非叶结点都有两个分支，每次决策都只有“是”和“不是”两个选项。
- CART 分类决策树则使用**基尼系数**来选择划分属性。

$$Gini(D) = \sum_{k=1}^{|Y|} \sum_{k^{'}=k} p_k p_k^{'} = 1 - \sum_{k=1}^{|Y|} p_k^2$$

- 基尼系数反映了从数据集 D 中随机抽取两个样本，其类别标记不一样的概率。**基尼系数越小，数据集的纯度越高**

$$Gini_index(D,a) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} Gini(D^v)$$

CART 算法

- 选择可以使得**基尼系数最小**的属性作为最优划分属性

$$a^* = \underset{a \in A}{\operatorname{argmin}}\, Gini_index(D, a)$$

- 计算“触感”属性下硬滑、软粘两种取值的基尼值

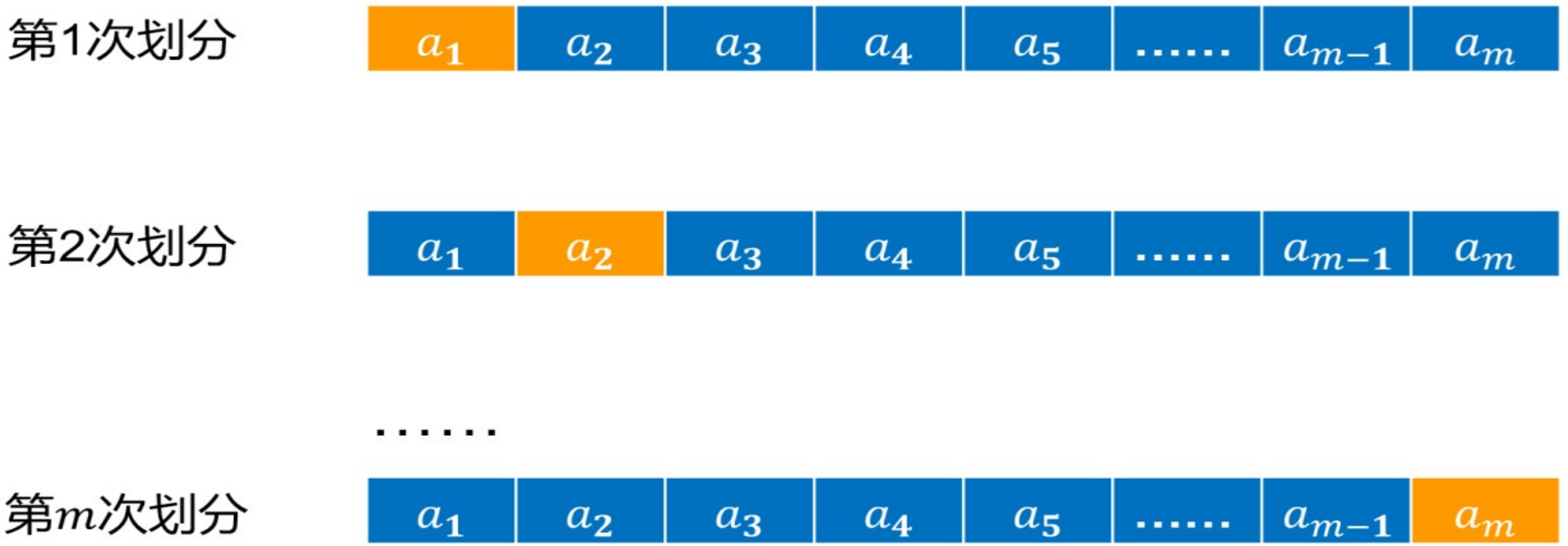
$$Gini(D, a^{\text{硬滑}}) = 1 - \left(\frac{3}{6}\right)^2 - \left(\frac{3}{6}\right)^2 = 0.5$$

$$Gini(D, a^{\text{软粘}}) = 1 - \left(\frac{2}{4}\right)^2 - \left(\frac{2}{4}\right)^2 = 0.5$$

$$Gini_index(D, \text{触感}) = \frac{6}{10} \times 0.5 + \frac{4}{10} \times 0.5 = 0.5$$

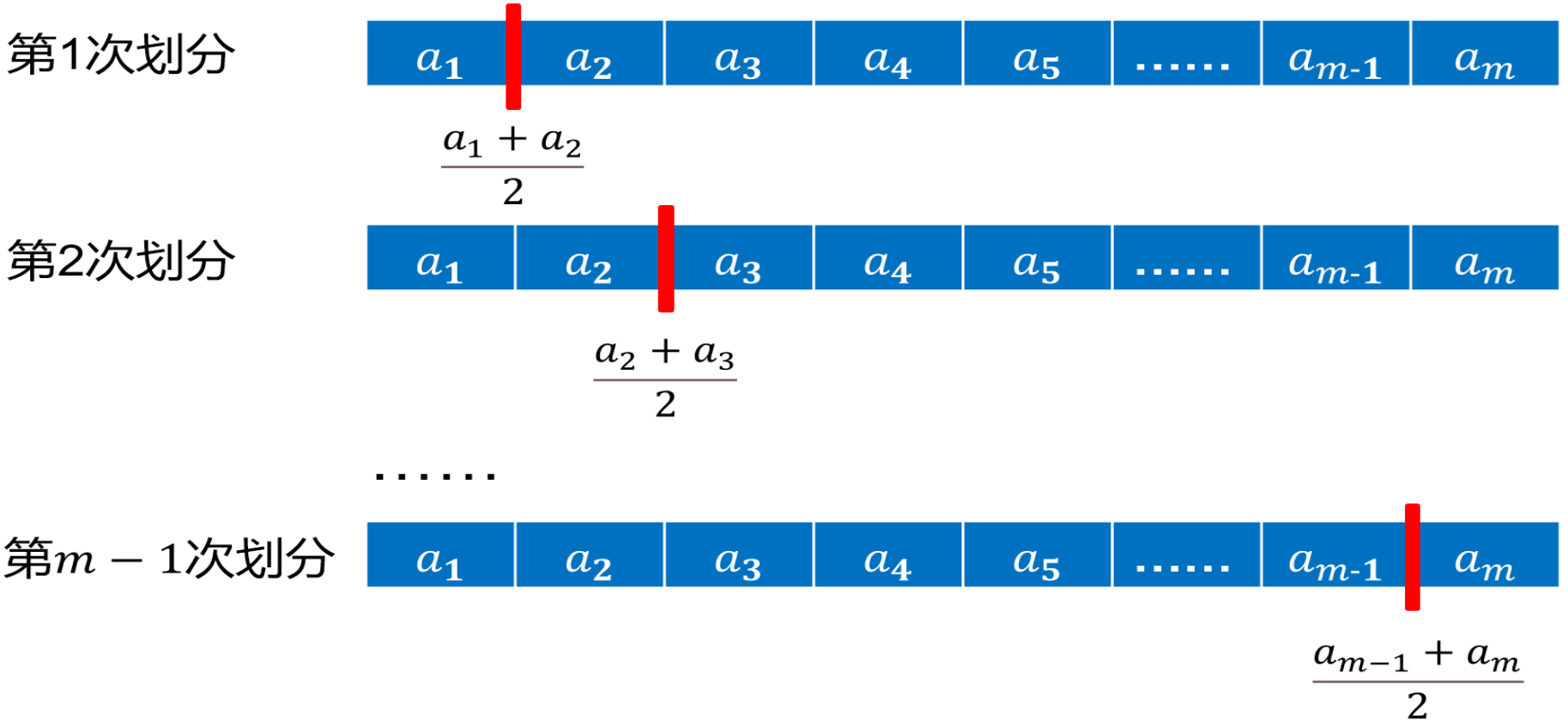
CART 算法

- 对于某些具有多个取值的属性（比方说“色泽”、“根蒂”、“脐部”等）而言，我们需要不断地进行二值划分



CART 算法

- 对于某些具有连续特征值的属性，我们也有方法进行二值划分



CART 算法

- CART 回归树需要遍历所有的特征，然后使用二分法划分数据子集，然后计算各个数据子集的总方差用于衡量**数据子集的混乱程度**，方差越小的数据子集“纯度”越高。最后我们选择**方差最小的属性**进行决策树的划分。最终的输出结果取决于各个子叶节点的输出均值。
- CART 回归树的剪枝流程如下
  - 计算每个节点的损失函数值（回归树用平方误差）。
  - 递归地从子叶节点开始向上遍历，并剪掉叶节点，然后判断损失函数的值是否减少。如果减少，则将父节点作为新的子叶节点。
  - 重复上述步骤，直到不能继续剪枝为止。

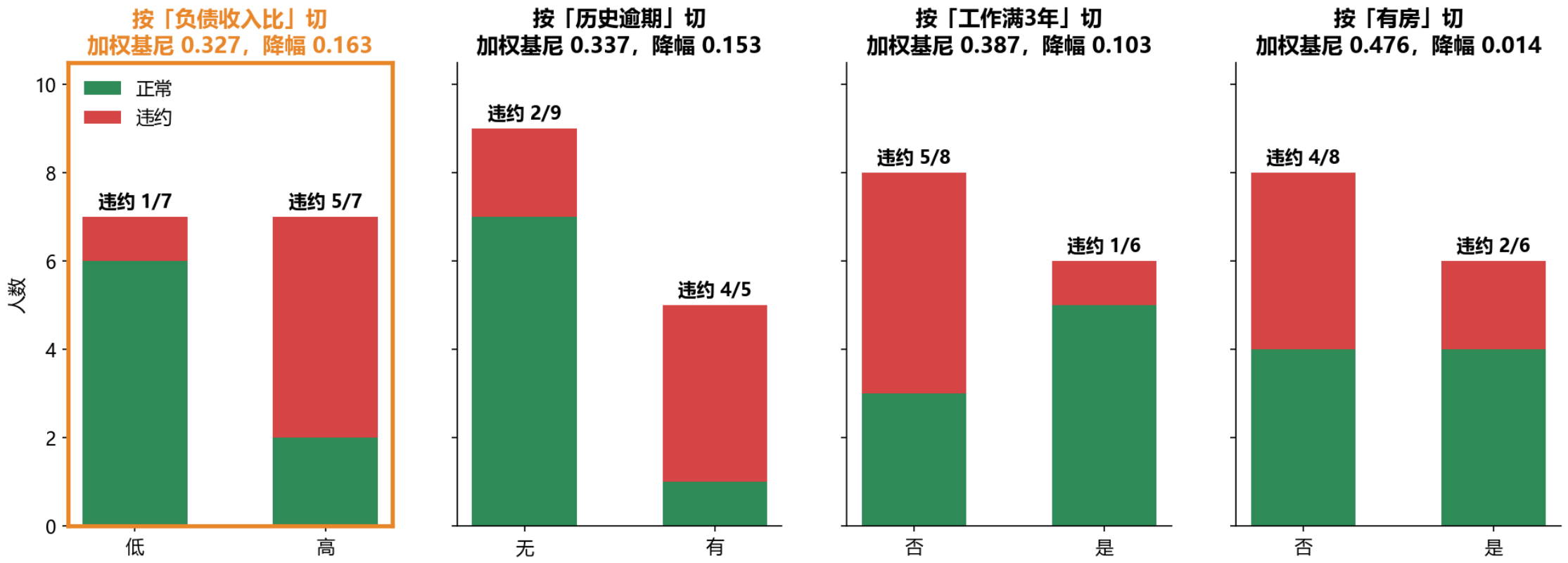
4. 决策树

- **优点：**
  - 可解释性强，计算容易。
  - 不需要对数据进行归一化。
  - 能够用于处理有缺失属性的样本
  - 可评估各个属性变量的重要性，自动剔除对生成决策树没有贡献的属性。
- **缺点：**
  - 忽略了数据之间的关联性
  - 容易过拟合，需要进行额外的剪枝操作
  - 不能用于处理流数据，对于新加入的样本，需要重新训练决策树，由此带来额外的计算开销。

先问哪个问题

四个候选问题各切一刀：负债收入比最干净，有房几乎没用。

图 1　四个候选问题，谁把 14 位借款人分得最干净（切之前基尼系数 0.490）

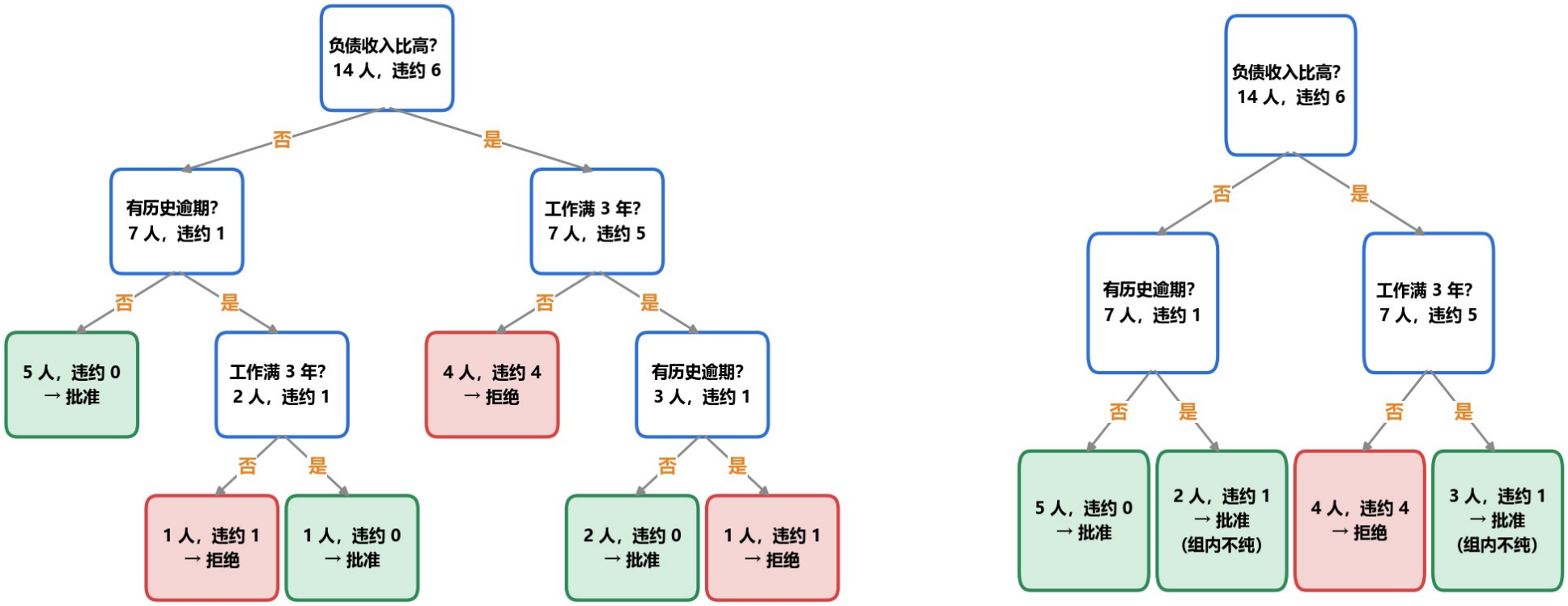


案例： 14 位历史借款人；绿色正常，红色违约（教学构造数据）。

完整树与剪枝树

不限深度： 6 个叶子，训练集 100%； max_depth=2： 4 个叶子，训练集 85.7%。

图 2 同一批 14 位借款人，长满的树与剪过的树（蓝框 = 问题，绿框 = 批准，红框 = 拒绝）

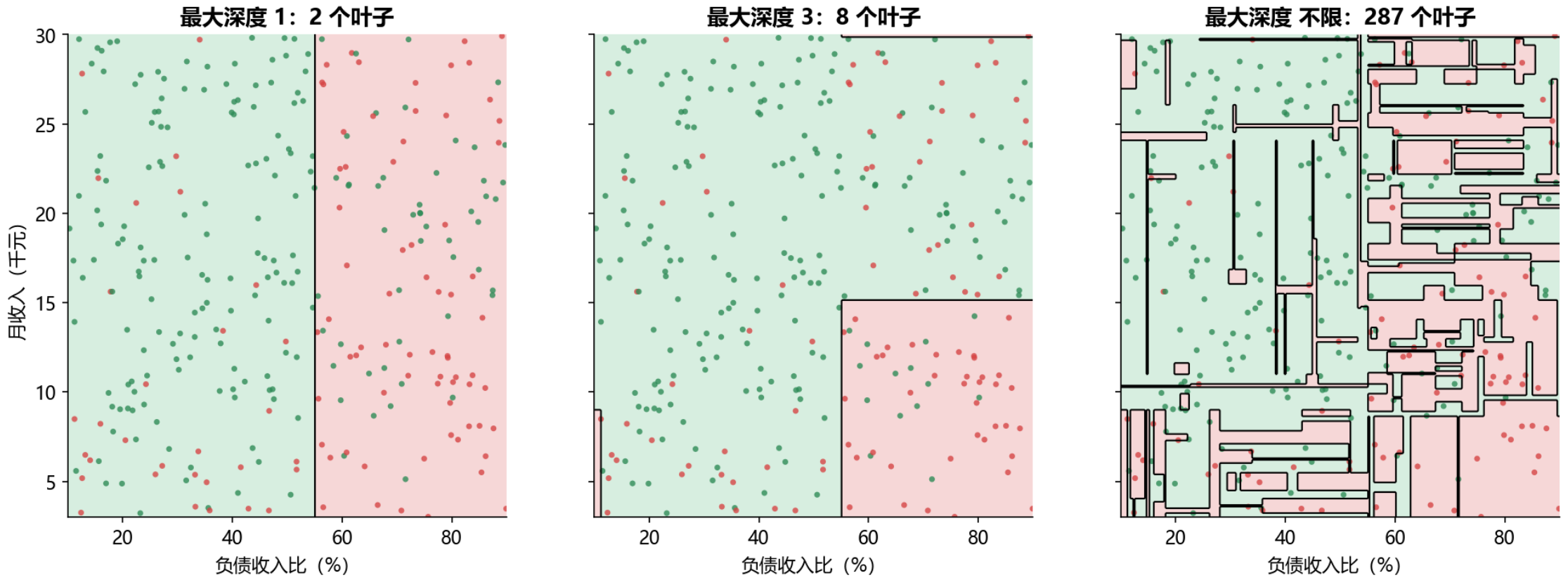


剪枝用少量训练误差换取更稳的规则；每个叶子至少有 2 人支撑。

决策树在切什么

每问一个“是否超过某个值”，就在平面上划一条横线或竖线。

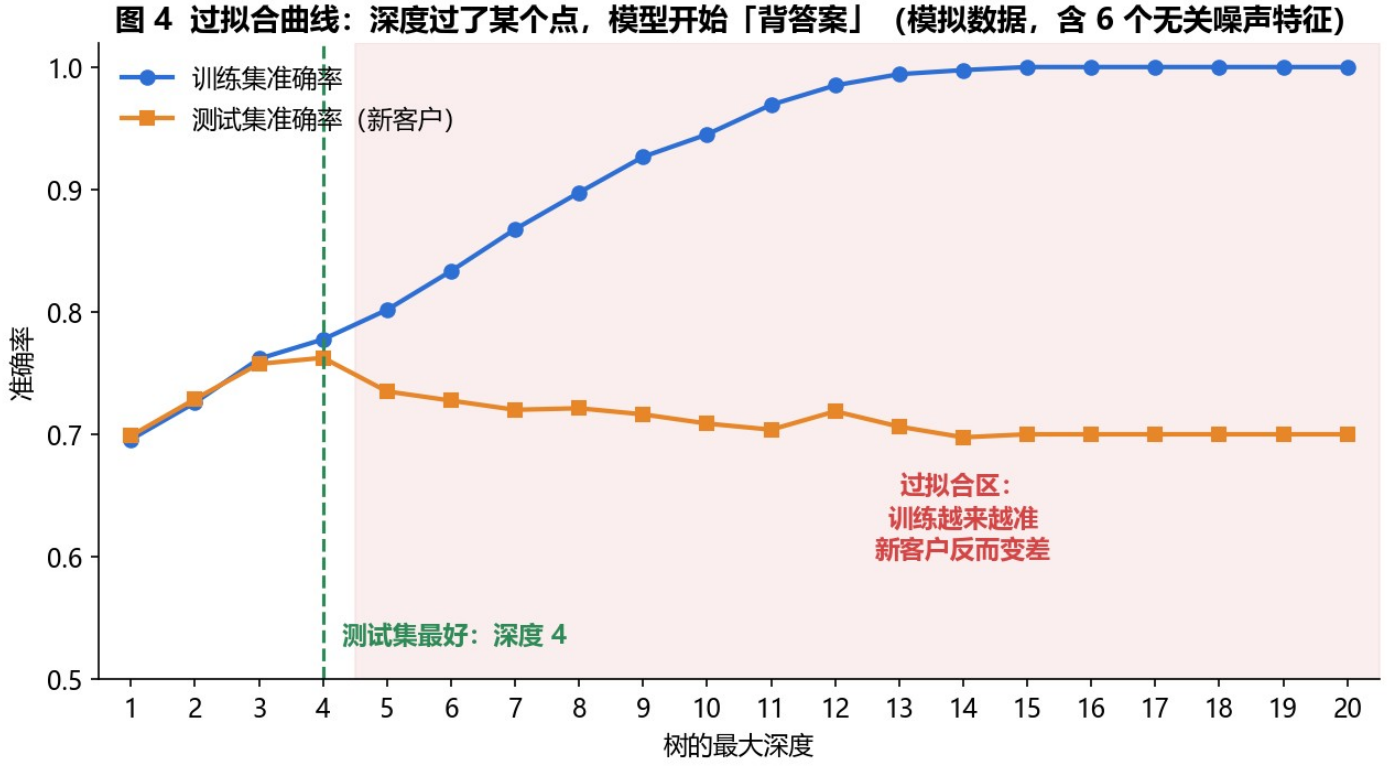
图 3　决策树把平面切成一块块矩形；越深切得越碎（模拟数据，绿区判正常，红区判违约）



深度 3 能切出“负债高并且收入低”的交互区域；不限深度会把平面切碎。

过拟合：训练集越准，新客户未必越好

蓝线训练集一路升到 100%，橙线测试集在深度 4 后下滑。



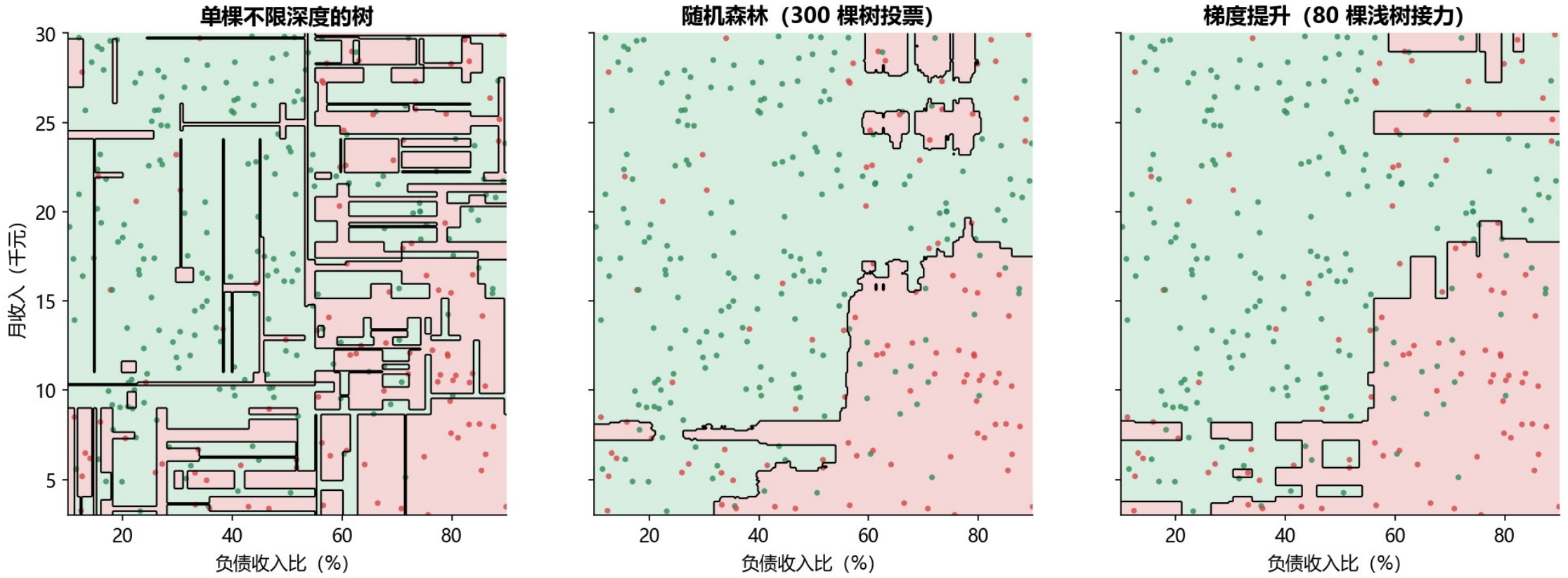
- 两条线分开的地方，就是模型开始“背答案”的地方。
- 训练集准确率下降，不等于模型变差；要看没见过的新客户。
- 剪枝和限制深度，就是把树停在测试集最好的位置附近。

模拟 2,000 位借款人， 60% 训练、 40% 测试，另加 6 个噪声特征。

一棵树，一片森林

随机森林和梯度提升把许多棵树的结果合在一起。

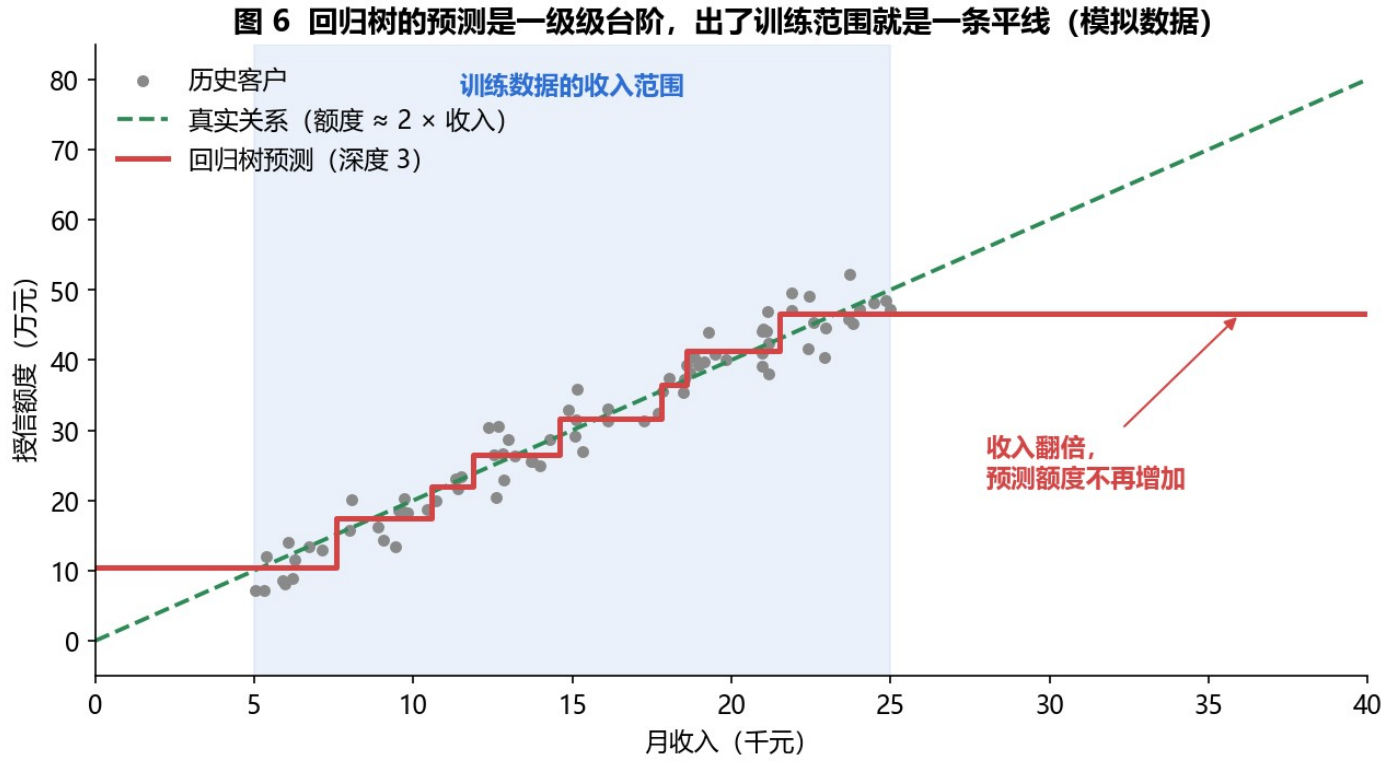
图 5　从一棵树到一片森林：边界更稳，但已无法画成一张规则图（模拟数据）



集成通常更准，但可解释性下降： 100 棵树的投票结果很难写成几条规则。

回归树不擅长外推

叶子输出一个数；训练区间外只能沿用最右边一级台阶。



- 区间内用一级级台阶逼近斜线，还算准。
- 区间外，比如月收入 4 万，预测额度停在约 47 万。
- 决策树只会在见过的情况里挑答案，不会顺着趋势往外画线。

模拟 80 位客户，真实额度约为月收入的 2 倍。

三模型对照

三种模型没有绝对优劣，关键是问题、数据形态和解释要求。

kNN

核心：看新样本像谁

训练：存样本，无参数

解释：和这几位客户像

关键风险：量纲、维数灾难

标准化：必须

朴素贝叶斯

核心：证据叠加概率

训练：一张概率表

解释：哪几条证据让概率升高

关键风险：独立性不成立

标准化：不需要

决策树

核心：一连串是或否的问题

训练：一棵规则树

解释：因为负债高且工作未满 3 年

关键风险：过拟合、不稳定

标准化：不需要

一句话： kNN 靠相似，朴素贝叶斯靠概率，决策树靠规则。

课堂讨论

任选两题，用本讲的概念回答。

01　如果把“客户身份证号”作为特征放进 ID3，会发生什么？ C4.5 怎么防止？

02　单棵决策树可解释， XGBoost 更准；银行风控该怎么选？

03　标准化前后 kNN 对同一位客户给出相反判决，业务上该信哪一个？

结合讲义与案例图，说明你的判断依据和适用条件。